# Deep Learning Interview Questions & Answers

> **What this covers:** ANN → How networks are trained → CNN → LeNet-5 → AlexNet → VGG-16/19 → GoogLeNet/Inception → ResNet/ResNet-50 → Transfer Learning → Classification / Detection / Segmentation → YOLO → RNN → LSTM/GRU → Seq2Seq → Attention → VAE → GAN → Debugging scenarios
> **Not covered in this round:** Transformers (we stop at attention, right where Transformers begin)

## How to use this document

Every answer follows the same pattern:

**Simple answer**

What you say first in the interview, in 2–3 plain sentences. If the interviewer moves on after this, you've done well.

Then a **detailed explanation** with examples, small calculations and diagrams, for when the interviewer says *"can you go deeper?"*

**⚠ Common mistake**

Something many candidates get wrong, or a detail that shows you really understand the topic.

**Interviewer may ask next**

Follow-up questions, with short answers.

**Tip:** interviewers often ask you to calculate something on the spot, such as an output size, a parameter count or an IoU score. Each formula comes with a worked example, and they're all collected in the Formula Cheat Sheet at the end.

**About the maths:** you don't need to memorise every formula. Each one is explained in words. Understand the idea first, and the formula will make sense.

## Key terms in plain English

Read this once before you start. These words appear everywhere.

| Term | Plain-English meaning |
|---|---|
| **Weight** | A number the network learns. It says how much one input matters. |
| **Bias** | An extra learned number added to a neuron's sum. It shifts the result up or down. |
| **Parameters** | All the weights and biases together: everything the network learns. |
| **Activation function** | A small function applied after each neuron's sum, such as ReLU or sigmoid. It lets the network learn curves, not just straight lines. |
| **Layer** | A group of neurons that all work on the same input at the same time. |
| **Hidden layer** | Any layer between the input and the output. |
| **Forward pass** | Feeding an input through the network to get a prediction. |
| **Loss** | One number that measures how wrong the prediction is. Lower is better. |
| **Gradient** | For each weight, which direction to change it (and by how much) to reduce the loss. |
| **Backpropagation** | The method that calculates the gradient for every weight, working backwards from the loss. |
| **Learning rate** | How big a step we take each time we update the weights. |
| **Optimizer** | The rule for updating weights using gradients (SGD, Adam, ...). |
| **Epoch** | One full pass through the whole training dataset. |

| Term | Plain-English meaning |
|---|---|
| **Batch** | A small group of samples processed together before one weight update. |
| **Logit** | The raw score a network outputs *before* it's turned into a probability. |
| **Overfitting** | The model memorises the training data and does badly on new data. |
| **Underfitting** | The model is too simple, or hasn't trained enough, to learn the pattern even on training data. |
| **Feature** | A useful piece of information about the input, such as an edge, a texture or a word. |
| **Feature map** | The output of one filter in a CNN: a grid showing where a pattern was found. |
| **Channel** | One "layer" of an image or feature map. RGB images have 3 channels. |
| **Inference** | Using a trained model to make predictions (no more learning). |
| **Pretrained model** | A model already trained by someone else on a large dataset, which you can reuse. |

### Roadmap


| Part | Sections |
|---|---|
| **A. Neural network basics** | 1. Why DL · 2. The neuron and MLP · 3. Activations · 4. Losses and outputs · 5. How training works · 6. Preventing overfitting |
| **B. Convolutional networks** | 7. CNN basics · 8. CNN shape and parameter calculations · 9. LeNet-5 · 10. AlexNet · 11. VGG · 12. GoogLeNet / Inception · 13. ResNet · 14. Comparing architectures · 15. Transfer learning and augmentation |
| **C. Vision tasks** | 16. Classification vs detection vs segmentation · 17. Object detection and YOLO |
| **D. Sequence models** | 18. Why sequence models · 19. RNN · 20. LSTM and GRU · 21. Seq2Seq · 22. Attention |
| **E. Generative models** | 23. Autoencoders and VAE · 24. GAN · 25. VAE vs GAN |
| **F. Interview practice** | 26. Debugging and real-world scenarios · 27. Rapid-fire · Formula cheat sheet · Revision table · Architecture map |


---

# 1. Why Deep Learning

## Q1. We already have Linear Regression, Logistic Regression, Decision Trees and Random Forests. Why did we need neural networks?

> 🔥 **Simple answer**
>
> Classical models need a human to decide **which features to give them**. Neural networks **learn the useful features by themselves** from raw data such as pixels, sound or text. For images, audio and language, designing features by hand is very hard, and features learned by the network work much better.

**An example.** Suppose you want to tell cats from dogs in a 224×224 photo. A Random Forest would see 150,528 separate pixel numbers. One pixel being "brown" tells you almost nothing. The meaning is in **how pixels are arranged**: edges form shapes, shapes form ears and eyes, and those form a face.

Before 2012, people hand-designed these features (methods with names like SIFT and HOG), then trained a classifier on top. Deep learning removed that manual step.

```
Classical ML:   raw data → features designed by a human → model → prediction
Deep Learning:  raw data → features learned by many layers → prediction
                           (everything is learned together)
```

Three things made deep learning work around 2012:

1. **Big datasets.** ImageNet had 1.2 million labelled images.
2. **GPUs.** Graphics cards made training large networks fast enough.
3. **Better training tricks.** ReLU, dropout and better weight initialisation, plus later BatchNorm and residual connections.

⚠ **Common mistake**

Don't say "deep learning is always better." For **table data** (rows and columns, like customer records), tree models such as XGBoost or LightGBM often do as well or better. They train faster and are easier to explain. Deep learning shines when the data has **structure**, such as images (space), audio (time) or text (language), and when there's **a lot** of it.

**Interviewer may ask next**

- **"When would you *not* use deep learning?"** Small table datasets, when you must explain every decision, when you have very little compute, or when you have very little labelled data and no pretrained model to start from.
- **"What is representation learning?"** The network learns to transform the input into a new form (a "representation") that makes the task easy. By the last layer, the classes are often easy to separate.

---

# 2. The Neuron and the MLP

## Q2. What exactly does a single neuron compute?

**Simple answer**

It multiplies each input by a weight, adds them up, adds a bias, and passes the result through an activation function: $a = f(w_1x_1 + w_2x_2 + \cdots + b)$. The weights say **which inputs matter**. The bias shifts the result. The activation lets the network learn **non-straight-line** patterns.

$$z = w_1x_1 + w_2x_2 + \cdots + w_nx_n + b, \qquad a = f(z)$$

where:

- $x_1 \ldots x_n$ are the inputs
- $w_1 \ldots w_n$ are the weights (learned)
- $b$ is the bias (learned)
- $f$ is the activation function (for example ReLU)
- $a$ is the neuron's output

**A small example.** Inputs $x = [2, 3]$, weights $w = [0.5, -1]$, bias $b = 1$:
$z = 0.5 \cdot 2 + (-1) \cdot 3 + 1 = -1$. With ReLU, $a = \max(0, -1) = 0$. The neuron stays "off" for this input.

A single neuron with a sigmoid activation is exactly **logistic regression**. It draws one straight line (or flat surface) to split the data into two sides.

**A whole layer** is just many neurons at once, which we write as one matrix multiplication:

$$H = f(XW + b)$$

If the input batch $X$ has shape (batch size × inputs), and $W$ has shape (inputs × neurons), then $H$ has shape (batch size × neurons). This is why GPUs help so much: they are very fast at matrix multiplication.

**Interviewer may ask next**

- **"Why do we need the bias?"** Without it, the line the neuron draws must always pass through zero. The bias lets it move, just like $c$ in $y = mx + c$.
- **"Is this how real brain neurons work?"** Only very loosely. The brain inspired the idea, but the maths is just a simple formula.

---

## Q3. Why can't a single-layer perceptron solve XOR? How does a hidden layer fix it?

**Simple answer**

A single perceptron can only draw **one straight line**. In XOR, the "1" points are on one diagonal and the "0" points are on the other, so no single straight line can separate them. A hidden layer lets the network draw **two lines** and combine them.

```
x1 x2 | XOR          x2
 0  0 |  0          1 |  ●(1)    ○(0)
 0  1 |  1            |
 1  0 |  1          0 |  ○(0)    ●(1)
 1  1 |  0            +----------------- x1
                        0        1
```

Try it: any straight line you draw will put at least one point on the wrong side.

**A real solution with 2 hidden neurons.** Here "step" outputs 1 if its input is positive, else 0.

- Hidden neuron 1 learns **OR**: $h_1 = \text{step}(x_1 + x_2 - 0.5)$
- Hidden neuron 2 learns **AND**: $h_2 = \text{step}(x_1 + x_2 - 1.5)$
- The output learns **"OR but not AND"**, which is XOR: $y = \text{step}(h_1 - h_2 - 0.5)$

| $x_1$ | $x_2$ | $h_1$ (OR) | $h_2$ (AND) | $y$ (XOR) |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 |
| 0 | 1 | 1 | 0 | 1 |
| 1 | 0 | 1 | 0 | 1 |
| 1 | 1 | 1 | 1 | 0 |

**The big idea:** the hidden layer **re-describes** the input as $(h_1, h_2)$. In this new description, the problem becomes easy to split with one line. That is what every hidden layer does: it transforms the data step by step until the final answer is easy.

**Common mistake**

This XOR limitation (shown by Minsky & Papert in 1969) is a big reason people lost interest in neural networks for years. Multi-layer networks could solve it, but nobody had a good way to *train* them until backpropagation became popular in 1986.

---

## Q4. What does a hidden layer actually give us? What is the Universal Approximation Theorem?

> 🔥 **Simple answer**
>
> Each hidden layer turns its input into a **new, more useful description** of the data. Stacking layers means each layer builds on the previous one's work. The Universal Approximation Theorem says that a network with **just one hidden layer**, if it has enough neurons, can approximate almost any function.

$$h_1 = f(W_1 x + b_1), \quad h_2 = f(W_2 h_1 + b_2), \quad \hat{y} = g(W_3 h_2 + b_3)$$

In words: layer 1 transforms the input, layer 2 transforms layer 1's output, and so on.

In an image network, the layers usually end up learning something like this:

```
Pixels → edges → corners and textures → small patterns → object parts → whole objects
```

Nobody programs these steps. They appear on their own during training, because each layer learns whatever makes the next layer's job easier.

> ⚠ **Common mistake**
>
> The theorem only says a good network **exists**. It does **not** say:
>
> - how many neurons you'd need (it could be a huge number),
> - that training will actually **find** the right weights,
> - that the network will work well on **new** data.
>
> So "one hidden layer is enough in theory" doesn't mean we use shallow networks in practice.

---

## Q5. If one wide layer can approximate anything, why go deep? And is deeper always better?

> 🔥 **Simple answer**
>
> Deep networks can **reuse** what earlier layers learned. An edge detector learned once can help build hundreds of shapes, and shapes can help build objects. A shallow network has to learn everything separately, so it needs far more neurons. But deeper isn't automatically better: very deep networks are harder to train, slower, and use more memory.

**An analogy.** Think of building with LEGO. A shallow network is like carving 1,000 different toys from separate blocks of wood. A deep network makes a small set of LEGO bricks (edges), combines them into parts (wheels, windows), and combines parts into toys. Much less work, because the pieces are shared.

```
Width (neurons per layer) → how many patterns you can detect at one stage
Depth (number of layers)  → how many steps of building-up you can do
```

| | Wider network | Deeper network |
|---|---|---|
| Power per parameter | Lower | Higher (reuses earlier features) |
| How easy to train | Easier | Harder without tricks like BatchNorm and skip connections |
| Speed on a GPU | Faster (one big calculation) | Slower (layers run one after another) |

Modern networks grow width, depth and image size **together** in balance. EfficientNet is a well-known example.

**Interviewer may ask next**

- **"What went wrong when people just added more layers?"** Beyond about 20 layers, plain networks got **worse even on training data**. This is the "degradation problem", which ResNet solved (see Q41).

---

## Q6. Count the parameters in an MLP for MNIST: 784 → 128 → 64 → 10.

**Simple answer**

Each neuron in a dense layer has one weight per input, plus one bias. So a layer going from $n_{in}$ inputs to $n_{out}$ neurons has $(n_{in} + 1) \times n_{out}$ parameters.

(MNIST images are 28×28 = 784 pixels, and there are 10 digit classes.)

| Layer | Calculation | Parameters |
|---|---|---|
| 784 → 128 | $784 \times 128 + 128$ | 100,480 |
| 128 → 64 | $128 \times 64 + 64$ | 8,256 |
| 64 → 10 | $64 \times 10 + 10$ | 650 |
| **Total** | | **109,386** |

**What to notice:** 92% of all parameters are in the **first layer**. Fully connected layers on big inputs need huge numbers of weights. For a real photo this gets out of hand, which is the main reason CNNs were invented (Q23).

```
model = keras.Sequential([
    keras.Input(shape=(784,)),
    layers.Dense(128, activation="relu"),
    layers.Dense(64, activation="relu"),
    layers.Dense(10, activation="softmax"),
])
model.summary()   # Total params: 109,386
```

---

# 3. Activation Functions

## Q7. Why do we need nonlinear activation functions?

**Simple answer**

Without them, stacking layers is pointless. Many linear layers in a row **combine into one single linear layer**, so a 100-layer network would be no smarter than a 1-layer one. The activation function adds the "bend" that lets the network learn curved, complex patterns.

Here is why. Two linear layers:

$$h_2 = W_2(W_1 x + b_1) + b_2 = \underbrace{(W_2 W_1)}_{\text{just one matrix}}\, x + \underbrace{(W_2 b_1 + b_2)}_{\text{just one bias}}$$

Multiplying two matrices gives another matrix, so the result is still one linear layer. Adding ReLU between the layers stops this collapse. The network can then build curved decision boundaries out of many small straight pieces.

**Interviewer may ask next**

- **"Isn't ReLU linear? It's a straight line on each side."** It's *piecewise* linear. The bend at 0 is enough. By combining many bends at different places, the network can approximate any curve.
- **"Does the output layer need an activation?"** Only the one the task needs: none for predicting a number, sigmoid or softmax for classification (see Q10).

---

## Q8. Compare Sigmoid, Tanh, ReLU, Leaky ReLU and GELU.

**Simple answer**

Sigmoid and tanh **flatten out** for large inputs, so their gradient becomes almost zero and deep networks stop learning. ReLU keeps a gradient of exactly 1 for positive inputs and is very cheap to compute. That's why it became the default. Its weakness is "dead" neurons, which Leaky ReLU and GELU fix.

| Activation | Formula | Output range | Good | Bad |
|---|---|---|---|---|
| Sigmoid | $\frac{1}{1+e^{-x}}$ | 0 to 1 | Output looks like a probability | Flattens out; slope at most 0.25 |
| Tanh | $\frac{e^{x}-e^{-x}}{e^{x}+e^{-x}}$ | –1 to 1 | Centred on zero | Still flattens out |
| ReLU | $\max(0, x)$ | 0 to ∞ | Simple, fast, slope 1 when positive | Neurons can "die" (always 0) |
| Leaky ReLU | $\max(0.01x, x)$ | –∞ to ∞ | Small slope for negatives, so no dead neurons | One extra setting to choose |
| GELU | $x \cdot \Phi(x)$ (a smooth ReLU) | ≈ –0.17 to ∞ | Smooth; used in modern models | A bit more computation |

**Why "slope at most 0.25" is a problem.** During training, gradients pass backwards through every layer. With sigmoid, each layer multiplies the gradient by **at most 0.25**. After 10 layers: $0.25^{10} \approx 0.000001$. The early layers receive almost no learning signal. ReLU multiplies by 1 (for active neurons), so the signal survives.

**Common mistake**

Sigmoid is **not useless**. It is still the right choice for:

- the **output layer** when the answer is yes/no, or when there are several yes/no labels,
- the **gates inside LSTM and GRU**, where we want a value between 0 and 1 meaning "how much to let through".

It's just a poor choice for **hidden layers** in deep networks.

---

## Q9. What is the "dying ReLU" problem?

**Simple answer**

If a ReLU neuron's input becomes negative for **every** example, it always outputs 0. Its gradient is also 0, so its weights never change again. The neuron is "dead" for good.

**How it happens.** A learning rate that's too large can push a neuron's bias far into the negative. After that, $w \cdot x + b$ is below zero for every input.

**How to fix it:**

- Use a smaller learning rate.
- Use **He initialisation** (designed for ReLU, see Q15).
- Use **Leaky ReLU, ELU or GELU**, which still have a small slope for negative inputs.
- Add **BatchNorm**, which keeps neuron inputs centred around zero.

**Interviewer may ask next**

- **"How would you spot dead neurons?"** Pass a batch of data through the network and check each layer. If some neurons output 0 for almost every example, they're dead.

---

## Q10. Which activation and loss should the output layer use?

**Simple answer**

It depends on **what you are predicting**. The output activation and the loss function always go together as a pair.

| Task | Output layer | Activation | Loss (Keras name) |
|---|---|---|---|
| Predict a number (regression) | 1 neuron | None (linear) | `mse`, `mae` |
| Yes/no (binary) | 1 neuron | Sigmoid | `binary_crossentropy` |
| Pick one of $K$ classes | $K$ neurons | Softmax | `categorical_crossentropy` (one-hot labels) or `sparse_categorical_crossentropy` (integer labels) |
| Several labels can be true at once (multi-label) | $K$ neurons | Sigmoid on each | `binary_crossentropy` |

**Multi-label is the row people get wrong.** If one photo can contain both a "dog" and "grass", don't use softmax. Softmax makes the classes compete, because the probabilities must add up to 1. Use a separate sigmoid for each label instead.

**Common mistake**

In Keras, if your last layer has **no activation** (it outputs raw scores, called logits), you must tell the loss `from_logits=True`. If you mix these up, for example softmax in the model *and* `from_logits=True`, training quietly goes wrong. Outputting logits and letting the loss handle them is actually the more numerically stable choice.

---

# 4. Loss Functions and Outputs

## Q11. Why do we use cross-entropy instead of MSE for classification?

**Simple answer**

Cross-entropy gives a **strong learning signal when the model is confidently wrong**. MSE combined with a sigmoid does the opposite: its gradient becomes tiny exactly when the model is very wrong, so learning stalls. Cross-entropy is also the mathematically "natural" loss for probabilities.

**Binary cross-entropy:**

$$\text{Loss} = -\left[y \log \hat{p} + (1-y) \log(1-\hat{p})\right]$$

where $y$ is the true label (0 or 1) and $\hat{p}$ is the predicted probability. In simple words: it's $-\log$ of the probability the model gave to the **correct** answer.

**A worked example.** The true answer is $y = 1$, but the model predicts $\hat{p} = 0.0025$. It's very confident and very wrong.

- **Cross-entropy** gradient is about $\hat{p} - y = -1.0$. That's a **big push** in the right direction.
- **MSE** gradient is about $-0.005$. **Almost no push at all.**

The difference comes from the sigmoid's slope, which is nearly zero at extreme values. MSE gets multiplied by that slope; cross-entropy cancels it out.

Cross-entropy also punishes confident mistakes hard: $-\log(0.0025) \approx 6.0$, while a confident correct answer costs only $-\log(0.9) \approx 0.1$.

**Interviewer may ask next**

- **"What's categorical cross-entropy?"** The same idea for many classes: $-\log$ of the probability given to the true class.
- **"What is label smoothing?"** Instead of a target of exactly 1 for the right class and 0 for the others, use something like 0.9 and spread 0.1 over the others. This stops the model from becoming over-confident, and it often helps on new data.

---

## Q12. What does softmax do, and why do we subtract the max before computing it?

**Simple answer**

Softmax turns a list of raw scores (logits) into **probabilities** that are all positive and add up to 1. Bigger scores get bigger probabilities. We subtract the largest score first because numbers like $e^{1000}$ are too big for a computer. Subtracting the same number from every score doesn't change the answer, but it avoids that overflow.

$$\text{softmax}(z)_k = \frac{e^{z_k}}{\sum_j e^{z_j}}$$

In words: take $e$ to the power of each score, then divide each by the total.

**Example:** scores $[2.0, 1.0, 0.1]$
→ $e^{\text{score}}$ = $[7.39, 2.72, 1.11]$, total = 11.21
→ probabilities = $[0.66, 0.24, 0.10]$

The exponential **exaggerates differences**. A score gap of just 1 makes one probability about 2.7× bigger than the other. That's why it's called a "soft" version of picking the maximum.

**Interviewer may ask next**

- **"What is temperature?"** You divide the scores by a number $T$ before softmax. $T < 1$ makes the model more confident (sharper), and $T > 1$ makes it less confident (flatter). It's used when sampling text from generative models.
- **"Are softmax outputs trustworthy probabilities?"** Not always. Deep networks are often over-confident. A simple fix called temperature scaling, tuned on validation data, can correct this.

---

# 5. How Training Works: Backpropagation and Optimization

## Q13. What is backpropagation actually doing?

**Simple answer**

Backpropagation works out **how much each weight contributed to the error**, using the chain rule from calculus. It starts at the loss and moves **backwards** layer by layer, reusing results as it goes, so it's fast. It doesn't change any weights itself. The optimizer (like SGD or Adam) uses those results to update the weights.

**An analogy.** A team project gets a bad grade. Backpropagation is like working backwards to figure out how much each person's work contributed to the bad grade. Then everyone adjusts in proportion to their share of the blame.

**A tiny worked example** with one neuron, $\hat{y} = \text{sigmoid}(w \cdot x + b)$. Take $x = 2$, $w = 0.5$, $b = 0$, and the true answer $y = 1$:

1. **Forward:** $z = 0.5 \times 2 = 1.0$ → $\hat{y} = 0.731$ → loss $= -\log(0.731) = 0.313$
2. **Backward:** the gradient at $z$ is $\hat{y} - y = -0.269$
3. **Chain rule:** the gradient for $w$ is $-0.269 \times x = -0.538$, and the gradient for $b$ is $-0.269$
4. **Update** (learning rate 0.1): $w = 0.5 - 0.1 \times (-0.538) = 0.554$

The new $w$ makes the next prediction a bit closer to 1. Training repeats this millions of times.

```
Forward:   input → layer 1 → layer 2 → … → prediction → loss    (save the in-between results)
Backward:  loss → gradients for the last layer → … → gradients for layer 1
Update:    new weight = old weight − learning rate × gradient
```

At each layer, the backward step multiplies by that layer's activation slope. Keep this in mind: it's where "vanishing gradients" come from.

**⚠ Common mistake**

The forward pass must **save** every layer's output so the backward pass can use it. That's why training uses much more GPU memory than prediction, and why bigger batches need more memory.

---

## Q14. What are vanishing and exploding gradients, and how do we fix them?

**Simple answer**

During backprop, the gradient is **multiplied once for each layer**. If those numbers are mostly less than 1, the gradient shrinks towards zero and the early layers stop learning. That's **vanishing**. If they're mostly bigger than 1, the gradient grows huge and training blows up into NaN (not-a-number). That's **exploding**.

Repeated multiplication changes numbers very quickly:

$$0.5^{20} \approx 0.000001 \quad \text{(vanishes)}, \qquad 1.5^{20} \approx 3{,}300 \quad \text{(explodes)}$$

| What you see | Vanishing | Exploding |
|---|---|---|
| Loss | Stops improving early | Jumps around, then becomes NaN |
| Gradients | Tiny in early layers | Very large |

| Fix | Helps with | Why it works |
|---|---|---|
| ReLU-type activations | Vanishing | Slope is 1 for active neurons |
| Good initialisation (He / Xavier) | Both | Starts the signal at a healthy size |
| Batch Normalization | Both | Re-scales values at every layer |
| Residual (skip) connections | Vanishing | Gives gradients a shortcut path (see Q42) |
| LSTM / GRU | Vanishing (over time) | Memory is updated by adding, not multiplying |
| Gradient clipping | Exploding | Caps how big a gradient can get |
| Smaller learning rate | Exploding | Smaller steps |

```
optimizer = keras.optimizers.Adam(learning_rate=1e-3, clipnorm=1.0)  # gradient clipping
```

---

## Q15. Why can't we initialize all weights to zero? What are Xavier and He initialization?

**Simple answer**

If all weights start the same, every neuron in a layer computes the same thing and gets the same update, so they **stay identical forever**. The layer behaves like a single neuron. So we start with small **random** weights. The size of those random numbers matters: too small and the signal fades, too large and it explodes.

**The rule of thumb:** a neuron that adds up many inputs should have smaller weights, so the total doesn't get too big. So the weight size depends on the number of inputs ($n_{in}$).

| Method | Weight variance | Use with | In Keras |
|---|---|---|---|
| Xavier / Glorot | $\frac{2}{n_{in}+n_{out}}$ | Sigmoid / tanh | `glorot_uniform` (**the default**) |
| He | $\frac{2}{n_{in}}$ | ReLU (ReLU switches off about half the neurons, so the weights are made a bit bigger to make up for it) | `he_normal` |

**⚠ Common mistake**

**Biases can start at zero.** The random weights already make the neurons different. One special case: the LSTM forget gate's bias is usually set to 1, so the memory starts out "remembering" (Keras does this by default).

---

## Q16. Batch gradient descent vs SGD vs mini-batch. How does batch size affect training?

**Simple answer**

**Batch GD** looks at the entire dataset before each update: accurate, but slow, and it may not fit in memory. **SGD** updates after every single example: very noisy and inefficient. **Mini-batch** (usually 32–512 examples) is the practical middle ground, and it's what everyone uses. The small amount of noise from mini-batches even helps the model generalise.

| | Small batch (e.g. 32) | Large batch (e.g. 4,096) |
|---|---|---|
| Gradient | Noisier | Smoother |

| | Small batch (e.g. 32) | Large batch (e.g. 4,096) |
|---|---|---|
| Updates per epoch | More | Fewer |
| GPU efficiency | Lower | Higher |
| Works well on new data? | Often better (noise helps) | Can be worse unless tuned |
| Memory needed | Less | More |

**Rule of thumb:** if you make the batch $k$ times bigger, make the learning rate about $k$ times bigger too, and warm it up slowly at the start.

**Words to know:** one **iteration** (or step) is one update using one batch. One **epoch** is one full pass over the data. With 50,000 examples and a batch size of 100, one epoch is 500 iterations.

---

## Q17. Explain Momentum, RMSProp and Adam. Which would you use?

**Simple answer**

- **Momentum** remembers the direction of past updates, so it speeds up in consistent directions and stops zig-zagging.
- **RMSProp** gives each weight its own step size. Weights with big, noisy gradients take smaller steps.
- **Adam = Momentum + RMSProp combined.** It's the safe default choice. Plain SGD with momentum is still popular for image models and sometimes gives slightly better final results.

**Momentum, as an analogy:** a ball rolling down a valley. It picks up speed along the valley floor and stops bouncing from side to side.

$$v_t = 0.9\, v_{t-1} + g_t, \qquad \theta_t = \theta_{t-1} - \eta\, v_t$$

($g$ = current gradient, $v$ = running "velocity", $\eta$ = learning rate, $\theta$ = the weights)

**RMSProp:** keep a running average of each weight's squared gradients ($s$) and divide the step by its square root.

$$\theta_t = \theta_{t-1} - \frac{\eta}{\sqrt{s_t} + \epsilon}\, g_t$$

A weight with large gradients gets a smaller step, and one with small gradients gets a larger step.

**Adam** keeps both a running average of gradients (like momentum, called $m$) and of squared gradients (like RMSProp, called $v$):

$$\theta_t = \theta_{t-1} - \eta\, \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

Typical defaults: learning rate 0.001, $\beta_1 = 0.9$, $\beta_2 = 0.999$.

**Interviewer may ask next**

- **"What is bias correction in Adam?"** $m$ and $v$ start at zero, so in the first few steps they're too small. Adam scales them up a bit at the start to fix this. The effect disappears after a while.
- **"What is AdamW?"** Adam with weight decay done properly. In plain Adam, the weight-decay penalty gets mixed into the adaptive step sizes and becomes weaker than intended. AdamW applies it separately. It's the standard choice today.

---

### Q18. How do you choose a learning rate? What are LR schedules and warmup?

**Simple answer**

The learning rate is the **most important setting** to get right. Too high and the loss jumps around or explodes. Too low and training is painfully slow. In practice we change it during training (a **schedule**): start larger to learn quickly, then make it smaller to settle into a good solution. Often there's a short **warmup** at the very start.

```
Loss over time:
  LR too high:   \_/\/\_/‾‾ NaN       (unstable, blows up)
  LR too low:    ‾‾‾‾‾\_______        (slow, gets stuck high)
  Good LR:       ‾\___________       (drops fast, then steadily improves)
```

| Schedule | What it does |
|---|---|
| Step decay | Divide the LR by 10 at fixed epochs |
| Cosine decay | Lower the LR smoothly towards 0 along a curve |
| ReduceLROnPlateau | Lower the LR when the validation loss stops improving |
| Warmup | Start with a tiny LR and increase it over the first few hundred steps, to avoid early instability |

**LR range test:** train briefly while slowly increasing the LR from very small to very large. Watch where the loss starts going up, and pick an LR about 10× smaller than that point.

```
callbacks = [
    keras.callbacks.ReduceLROnPlateau(monitor="val_loss", factor=0.5, patience=3),
    keras.callbacks.EarlyStopping(monitor="val_loss", patience=8, restore_best_weights=True),
]
```

---

## 6. Preventing Overfitting: Regularization and Normalization

### Q19. What tools do we have to fight overfitting in a deep network?

**Simple answer**

Overfitting means the model memorises the training data instead of learning general patterns. The best fixes, roughly in order: **get more data or use data augmentation**, **start from a pretrained model**, add **dropout** and **weight decay**, use **early stopping**, and if needed **make the model smaller**. Then check that the gap between training and validation scores shrinks.

| Technique | What it does, in plain words |
|---|---|
| More data / data augmentation | Show the model more variety, so memorising is harder |
| Transfer learning | Start from a model that already learned from millions of images |
| Dropout | Randomly switch off neurons during training, so none can be relied on too much |
| L2 / weight decay | Penalise large weights, which keeps the model simpler |
| Early stopping | Stop training when validation results stop improving |
| Batch Normalization | Adds a small amount of helpful noise |

| Technique | What it does, in plain words |
|---|---|
| Smaller model | Less capacity to memorise |
| Label smoothing, Mixup, CutMix | Discourage over-confident memorising |

> ⚠ **Common mistake**
>
> Before adding regularization, check that it's **really** overfitting:
>
> - near-duplicate images, or the same patient, in both train and validation (leakage),
> - train and validation data coming from different sources,
> - preprocessing applied differently to train and validation.
>
> Regularization won't fix any of these.

---

## Q20. How does dropout work, and why does it behave differently at inference?

> 🔥 **Simple answer**
>
> During training, dropout **randomly switches off** a fraction of neurons (for example 50%) on every step. The network can't depend on any single neuron, so it learns backup ways to recognise things. It's like training many slightly different networks that share weights. When making real predictions (inference), dropout is **turned off** and all neurons are used.

**An analogy.** A football team where random players sit out each practice. Everyone has to learn to play well with any combination of teammates, so the team becomes more robust.

**The scaling detail.** If half the neurons are off during training but all are on at prediction time, the next layer would suddenly receive about twice as much signal. To fix this, during training the remaining active neurons are scaled up by $\frac{1}{1-p}$ (×2 when $p = 0.5$). Then nothing needs to change at prediction time. Keras does this automatically.

```
layers.Dense(256, activation="relu"),
layers.Dropout(0.5),     # 0.5 = fraction of neurons to DROP
```

Common rates: 0.5 for big dense layers, 0.1–0.3 for conv layers.

> ⚠ **Common mistake**
>
> - In Keras, `Dropout(0.2)` means **drop** 20%. (The original paper used $p$ to mean the probability of *keeping* a neuron.)
> - With dropout, training loss is often **higher** than validation loss, because training is deliberately made harder. That's normal, not a bug.

---

## Q21. What does Batch Normalization do? How does it behave during training vs inference?

**Simple answer**

BatchNorm re-scales the values inside the network so that, for each channel, they have an **average of 0 and a spread of 1**, calculated over the current batch. Then it applies a learned scale ($\gamma$) and shift ($\beta$). This makes training faster and more stable and allows higher learning rates. During prediction there may be no batch, so it uses **running averages** saved during training instead.

$$\hat{x} = \frac{x - \text{mean}_{batch}}{\sqrt{\text{variance}_{batch} + \epsilon}}, \qquad y = \gamma\,\hat{x} + \beta$$

In words: subtract the batch average, divide by the batch spread, then let the network re-scale and shift with its own learned $\gamma$ and $\beta$. ($\epsilon$ is a tiny number that avoids dividing by zero.)

Why learn $\gamma$ and $\beta$? So the network can undo the normalisation if that works better. BatchNorm never limits what the layer can learn.

| | During training | During prediction |
|---|---|---|
| Average and spread come from | The current batch | Running averages saved during training |
| Does one image's output depend on the others in the batch? | Yes | No |

**Why it helps:** it keeps values in a healthy range at every layer. This makes the training "landscape" smoother, so bigger, steadier steps are possible.

**⚠ Common mistake**

- With **very small batches** (under about 16, common in detection and segmentation), the batch statistics are noisy. Use **GroupNorm** or **LayerNorm** instead.
- **Keras fine-tuning issue:** when reusing a pretrained model, call it as `base_model(x, training=False)`. This keeps its BatchNorm layers using their saved statistics. Otherwise your small dataset overwrites them and damages the pretrained model.
- A conv layer followed by BatchNorm doesn't need its own bias (`use_bias=False`), because BatchNorm's $\beta$ already does that job.

**Interviewer may ask next**

- **"BatchNorm vs LayerNorm?"** BatchNorm averages **across the batch** for each feature. LayerNorm averages **across the features** within each single example. LayerNorm doesn't depend on batch size, which is why RNNs and Transformers use it.

---

### Q22. What is the difference between L2 regularization and weight decay, and how does early stopping act as regularization?

**Simple answer**

**L2 regularization** adds a penalty for large weights to the loss, which pushes weights to stay small and keeps the model smoother. **Weight decay** directly shrinks every weight a little at each step. With plain SGD the two are the same thing; with Adam they're not, which is why AdamW exists. **Early stopping** stops training before the weights can grow too large and fit noise, so it has a similar effect.

```
layers.Dense(128, activation="relu", kernel_regularizer=keras.regularizers.l2(1e-4))
```

With early stopping, always set `restore_best_weights=True`. Otherwise you keep the weights from the **last** epoch, not the **best** one.

---

## 7. CNN Basics

### Q23. Why can't we just flatten an image and feed it into a fully connected network?

**Simple answer**

There are two problems. **Far too many weights:** a 224×224 colour image has 150,528 numbers, so a single dense layer with 1,000 neurons would need about 150 million weights. **It throws away the layout:** flattening forgets which pixels are next to each other, and a pattern learned in one corner has to be learned again separately for every other position. CNNs fix both problems.

$$224 \times 224 \times 3 = 150{,}528 \text{ inputs} \;\Rightarrow\; 150{,}528 \times 1{,}000 \approx 150 \text{ million weights}$$

Compare that with a conv layer of 64 small 3×3 filters on the same image, which has only **1,792** parameters, and works on images of any size.

CNNs build in three common-sense **assumptions about images**:

| Assumption | In plain words | How a CNN does it |
|---|---|---|
| Locality | Pixels close together are related; far-apart pixels usually aren't (at first) | Small filters that look at a small patch (3×3, 5×5) |
| Weight sharing | A useful pattern, such as an edge, is useful anywhere in the image | The **same** filter slides across every position |
| Hierarchy | Complex things are made of simpler things | Stacking layers so each one sees a bigger area |

**⚠ Common mistake**

"CNNs have fewer parameters" is only half the answer. The real point is that a CNN's design **matches how images work**, so it learns from much less data. A plain dense network *could* learn the same thing in theory, but it would need far more examples to work it out on its own.

---

## Q24. What does a convolution filter compute, and what does it learn?

**Simple answer**

A filter is a small grid of learned numbers (for example 3×3) that **slides across the image**. At each position, it multiplies its numbers by the pixels underneath and adds them up. The result is high where that patch of the image **looks like** the filter's pattern. Nobody sets the filter values by hand. The network learns them, and many end up detecting edges, textures and shapes.

### A hand-made example: a vertical-edge detector

```
Filter          Patch with an edge            Flat patch
−1  0  +1        10  10  200                   50  50  50
−2  0  +2        10  10  200                   50  50  50
−1  0  +1        10  10  200                   50  50  50

Result: big number (760) — edge found!         Result: 0 — no edge
```

**An important detail: filters cover all channels.** On a colour (RGB) image, a "3×3 filter" is really **3×3×3**, one 3×3 slice per colour channel. On a feature map with 64 channels, a 3×3 filter is **3×3×64**. Each filter adds everything up into **one** output number per position, so each filter produces **one** output channel.

What filters tend to learn at different depths:

| Depth | Typical patterns |
|---|---|
| First layer | Edges at different angles, colour blobs |
| Middle layers | Corners, textures, simple repeated patterns |
| Deeper layers | Parts of objects: eyes, wheels, letters, fur |
| Final layers | Whole objects |

**Interviewer may ask next**

- **"Is it really convolution?"** Strictly speaking, libraries compute *cross-correlation* (the filter isn't flipped). Since the filter is learned anyway, it makes no difference: the network would simply learn the flipped version.

---

## Q25. What is a feature map? How do the number of filters and channels relate?

**Simple answer**

A feature map is the output of **one filter** across the whole image: a grid showing **where** that filter's pattern was found. A conv layer with 64 filters produces 64 feature maps, stacked together as 64 channels. Those 64 channels then become the input to the next layer.

```
Input 32×32×3 ──[ 64 filters, each 3×3×3 ]──▶ Output 32×32×64   (64 feature maps)
                                                      │
Next layer   ──[ 128 filters, each 3×3×64 ]──▶ Output 32×32×128
```

A pattern you'll see in almost every CNN: as we go deeper, the **image gets smaller** but has **more channels**.

```
224×224×3 → 112×112×64 → 56×56×256 → 28×28×512 → 14×14×1024 → 7×7×2048
less "where it is" ←————————————————————————————→ more "what it is"
```

We give up exact position in exchange for richer meaning. Doubling the channels each time the size halves keeps the amount of computation per layer roughly even.

---

## Q26. What are padding and stride? What do "valid" and "same" padding mean?

**Simple answer**

**Stride** is how far the filter moves each step. Stride 2 skips every other position, which halves the output size. **Padding** adds a border of zeros around the image so the filter can also sit on the edge pixels. **"Valid"** means no padding, so the output shrinks. **"Same"** means just enough padding that, with stride 1, the output stays the same size as the input.

| Setting (32×32 input, 3×3 filter) | Output size | What happens |
|---|---|---|
| `padding="valid"`, stride 1 | 30×30 | Shrinks by 2 |
| `padding="same"`, stride 1 | 32×32 | Same size (1 pixel of padding on each side) |
| `padding="same"`, stride 2 | 16×16 | Halved |

**Why padding matters:**

1. Without it, the image shrinks every layer. Ten 3×3 layers turn 32×32 into 12×12.
2. Without it, edge pixels are seen by fewer filter positions, so their information is used less.

**Why stride matters:** stride 2 is a learnable way to shrink the image (an alternative to pooling), and it makes all later layers cheaper.

---

## Q27. What is pooling? Max vs average pooling, and what is Global Average Pooling?

**Simple answer**

Pooling **shrinks** each feature map by summarising small windows. **Max pooling** keeps the biggest value in each window; **average pooling** keeps the average. It has **no learnable parameters**, reduces computation, and makes the network a little less sensitive to small shifts. **Global Average Pooling (GAP)** averages each entire feature map down to **one number**.

```
Max pool 2×2 (stride 2):

 1  3 | 2  1
 4  6 | 5  0          6  5
------+------    ⟶
 2  1 | 0  3          8  3
 7  8 | 1  2
(take the biggest number in each 2×2 block)
```

| | Max pooling | Average pooling |
|---|---|---|
| Question it answers | "Was this pattern found anywhere here?" | "How much of this pattern is here overall?" |
| Common use | Between conv layers | At the very end (global) |

**Why GAP replaced big dense layers at the end of CNNs:** in VGG, flattening a 7×7×512 output into a dense layer of 4,096 neurons costs about **103 million** parameters. GAP on a 7×7×2048 output just gives 2,048 numbers, at **zero** parameters. That means less overfitting, and the network can accept any image size.

**Interviewer may ask next**

- **"Do modern networks still use pooling?"** Less in the middle; many use stride-2 convolutions instead. GAP at the end is used almost everywhere.

---

## Q28. What is the difference between translation equivariance and translation invariance? Which does a CNN have?

**Simple answer**

**Equivariance:** if the object moves in the image, its features **move the same way** in the feature map. Convolution has this. **Invariance:** if the object moves, the **output doesn't change at all**. For classification we want this: "it's a cat" wherever the cat is. Convolution gives equivariance, and pooling (especially global pooling at the end) turns it into invariance.

```
Convolution:        cat moves right → cat features move right      (equivariant)
Conv + global pool: cat moves right → still "cat"                  (invariant)
```

**Common mistake**

CNNs are **not** naturally robust to **rotation or size changes**. A network trained only on upright digits may fail on rotated ones. We handle that with **data augmentation** (showing rotated and scaled examples during training), not with convolution itself.

---

## Q29. What is the receptive field, and how does it grow through a network?

**Simple answer**

The receptive field is **how much of the original image** a neuron can "see". A neuron after one 3×3 conv sees a 3×3 patch. Each extra layer makes this area bigger, and pooling or strides make it grow faster. Deep layers need large receptive fields to see whole objects.

**Simple case (stride 1, no pooling):** each extra 3×3 layer adds 2 pixels.

```
1 layer of 3×3  → sees 3×3
2 layers        → sees 5×5
3 layers        → sees 7×7
```

**General formula**, where $j$ is the total stride so far ("how far apart neighbouring neurons are in the original image"):

$$\text{new RF} = \text{old RF} + (k-1) \times j, \qquad \text{new } j = j \times \text{stride}$$

**Worked example** (start: RF = 1, $j$ = 1):

| Layer | RF | $j$ |
|---|---|---|
| Conv 3×3 | 1 + 2×1 = **3** | 1 |
| Conv 3×3 | 3 + 2×1 = **5** | 1 |
| MaxPool 2×2, stride 2 | 5 + 1×1 = **6** | 2 |
| Conv 3×3 | 6 + 2×2 = **10** | 2 |
| Conv 3×3 | 10 + 2×2 = **14** | 2 |

Notice that after pooling, each 3×3 conv adds 4 pixels instead of 2. This is why two 3×3 layers equal one 5×5, and three equal one 7×7, as used in VGG (Q37).

**Interviewer may ask next**

- **"How else can you grow the receptive field cheaply?" Dilated convolutions** spread the filter out with gaps. A 3×3 filter with dilation 2 covers a 5×5 area but still has only 9 weights. They're popular in segmentation.

---

## Q30. What does a 1×1 convolution do? Isn't it just multiplying by a constant?

**Simple answer**

On a single-channel image, yes. But feature maps have **many channels**. A 1×1 conv looks at **all the channels at one pixel** and combines them, like a tiny dense layer applied at every pixel. It changes the **number of channels** without changing the width or height. It's mainly used to **reduce channels and save computation**.

```
28×28×256 ──[ 64 filters of size 1×1×256 ]──► 28×28×64
Each output pixel = a learned mix of the 256 channel values at that same pixel.
```

Parameters: $(256 + 1) \times 64 = 16{,}448$.

Where you'll see it:

| Architecture | What the 1×1 conv does |
|---|---|
| GoogLeNet / Inception | Shrinks channels before expensive 3×3 and 5×5 filters (Q39) |
| ResNet-50 | Shrinks channels → 3×3 → expands channels again (Q44) |
| MobileNet | Mixes channels after a cheap per-channel filter |
| Segmentation heads | Turns features into one score per class at each pixel |

---

# 8. CNN Shape and Parameter Calculations

> These are some of the most common whiteboard questions. Practise them until they feel easy.

## Q31. What's the formula for the output size of a convolution? Work an example.

**Simple answer**

$$O = \frac{W - K + 2P}{S} + 1$$

where $W$ = input size, $K$ = filter size, $P$ = padding, $S$ = stride (round down if it's not a whole number). The output **depth** is simply the **number of filters**.

**Example 1, AlexNet's first layer:** 227×227×3 input, 96 filters of 11×11, stride 4, no padding.

$$O = \frac{227 - 11 + 0}{4} + 1 = 54 + 1 = 55 \;\Rightarrow\; \text{output } 55 \times 55 \times 96$$

**Example 2, ResNet's first layer:** 224×224×3 input, 64 filters of 7×7, stride 2, padding 3.

$$O = \frac{224 - 7 + 6}{2} + 1 = 111.5 \to 111, \; +1 = 112 \;\Rightarrow\; 112 \times 112 \times 64$$

**Example 3, checking "same" padding:** 3×3 filter, stride 1, padding 1 → $O = W - 3 + 2 + 1 = W$. The size stays the same. ✓

The same formula works for pooling. A 2×2 max pool with stride 2 on a 112×112 input gives 56×56.

**⚠ Common mistake**

The AlexNet paper says the input is **224**×224, but $(224 - 11)/4 + 1 = 54.25$ isn't a whole number. The maths only works with **227**. Interviewers sometimes use this to check whether you actually calculate or just remember numbers.

---

## Q32. How many parameters does a convolutional layer have? Work an example.

**Simple answer**

$$\text{Params} = (K \times K \times C_{in} + 1) \times C_{out}$$

Each filter has $K \times K \times C_{in}$ weights plus 1 bias, and there are $C_{out}$ filters. Notice that the **image size isn't in the formula**. The same filter is reused everywhere (weight sharing).

**Example:** a 3×3 conv from 64 channels to 128 channels:

$$(3 \times 3 \times 64 + 1) \times 128 = 577 \times 128 = 73{,}856$$

**Computation (multiply-adds) does depend on image size**, because the filter is applied at every output position:

$$\text{Multiply-adds} = H_{out} \times W_{out} \times C_{out} \times K \times K \times C_{in}$$

For the same layer on a 56×56 output, that's about **231 million** multiply-adds.

**A small full network** (32×32×3 input, like CIFAR-10):

| Layer | Output shape | Parameters |
|---|---|---|
| Conv 3×3, 32 filters, same | 32×32×32 | $(27 + 1) \times 32 = 896$ |
| MaxPool 2×2 | 16×16×32 | 0 |
| Conv 3×3, 64 filters, same | 16×16×64 | $(288 + 1) \times 64 = 18{,}496$ |

| Layer | Output shape | Parameters |
|---|---|---|
| MaxPool 2×2 | 8×8×64 | 0 |
| Flatten | 4,096 | 0 |
| Dense 128 | 128 | $4096 \times 128 + 128 = 524{,}416$ |
| Dense 10 | 10 | $1{,}290$ |
| **Total** | | **545,098** |

**What to notice:** that one dense layer holds **96% of all parameters**. If you replace Flatten with GlobalAveragePooling2D (giving 64 numbers), it drops to just $64 \times 128 + 128 = 8{,}320$.

---

### Q33. What is a depthwise-separable convolution, and why is it so much cheaper?

**Simple answer**

It splits a normal convolution into two cheaper steps. **Step 1 (depthwise):** filter each channel **separately**, which handles the spatial pattern. **Step 2 (pointwise):** use a 1×1 conv to **mix the channels**. For 3×3 filters this is about **8-9 times cheaper**, with only a small loss in accuracy. MobileNet is built from these.

| | Parameters (ignoring biases) |
|---|---|
| Normal conv | $K^2 \times C_{in} \times C_{out}$ |
| Depthwise-separable | $K^2 \times C_{in} + C_{in} \times C_{out}$ |

**Example:** 3×3 conv, 128 → 256 channels.

- Normal: $9 \times 128 \times 256 = 294{,}912$
- Separable: $9 \times 128 + 128 \times 256 = 33{,}920$ → **8.7× fewer**

```
layers.SeparableConv2D(256, 3, padding="same", activation="relu")
```

---

## 9. LeNet-5

### Q34. Walk me through LeNet-5. Why is it historically important?

**Simple answer**

LeNet-5 (Yann LeCun, 1998) read handwritten digits on bank cheques. It set up the basic recipe that **every CNN since has followed**: conv → pool → conv → pool → dense layers, trained end to end with backpropagation. It was tiny, with only about **60,000 parameters**.

| Layer | What it does | Output size |
|---|---|---|
| Input | Grayscale digit | 32×32×1 |
| C1 | 6 filters, 5×5 | 28×28×6 |
| S2 | Average pooling 2×2 | 14×14×6 |
| C3 | 16 filters, 5×5 | 10×10×16 |
| S4 | Average pooling 2×2 | 5×5×16 |
| C5 | 120 filters, 5×5 (works like a dense layer here) | 1×1×120 |

| Layer | What it does | Output size |
|---|---|---|
| F6 | Dense layer | 84 |
| Output | 10 digits | 10 |

How it differs from modern CNNs:

- It used **tanh/sigmoid-style** activations, not ReLU.
- It used **average pooling**, not max pooling.
- It was built for small grayscale images.

**Interviewer may ask next**

- **"If CNNs existed in 1998, why did they only take off in 2012?"** There wasn't enough data or computing power to scale them up, sigmoid/tanh made deeper networks hard to train, and other methods (SVMs with hand-designed features) were competitive. All three changed by 2012.

---

## 10. AlexNet

### Q35. If LeNet already used CNNs, what made AlexNet such a breakthrough?

**Simple answer**

AlexNet (2012) won the ImageNet competition with a **15.3% top-5 error**, while second place had **26.2%**. That huge gap convinced the whole field to switch to deep learning. The basic idea was LeNet's. What was new was the **scale** (60 million parameters, 1.2 million images, trained on GPUs) and the **training tricks** that made that scale work: ReLU, dropout and data augmentation.

**Structure:** 5 conv layers + 3 dense layers, about 60 million parameters.

```
227×227×3
 → Conv 11×11, 96 filters, stride 4 → 55×55×96 → MaxPool → 27×27×96
 → Conv 5×5, 256                   → 27×27×256 → MaxPool → 13×13×256
 → Conv 3×3, 384 → Conv 3×3, 384 → Conv 3×3, 256 → MaxPool → 6×6×256
 → Dense 4096 → Dense 4096 → Dense 1000 (softmax)
```

| What it introduced or popularised | Why it mattered |
|---|---|
| **ReLU** | Trained about 6× faster than tanh |
| **GPU training** | Two GPUs with 3 GB each; the model was split across them. Training took about 5–6 days |
| **Dropout (0.5)** | Stopped the huge dense layers from overfitting |
| **Data augmentation** | Random crops, horizontal flips and colour changes created more training variety |
| **Overlapping max pooling** | Slightly better accuracy |
| **Local Response Normalization** | A normalisation trick that later turned out not to help much; BatchNorm replaced it |

⚠ **Common mistake**

AlexNet didn't invent ReLU, dropout, CNNs or GPU training. Its achievement was showing that **all of them together, at large scale, on a big dataset** beat the old methods by a mile. Saying "architecture + data + compute + training tricks together" is the mature answer.

⍰ **Interviewer may ask next**

- **"Where are most of AlexNet's parameters?"** In the dense layers. The first dense layer alone has $6 \times 6 \times 256 \times 4096 \approx 37.7$ million, more than 60% of the model.
- **"What is top-5 error?"** A prediction counts as correct if the right answer is among the model's **top 5 guesses**. This is used because many ImageNet photos contain several objects.

---

## 11. VGG-16 and VGG-19

### Q36. What was the core idea behind VGG? How do VGG-16 and VGG-19 differ?

🔥 **Simple answer**

VGG (Oxford, 2014) tested a simple idea: **use only small 3×3 filters, and go deeper.** It worked: depth improved accuracy a lot. Its simple, repeating design made it very popular for reusing in other projects. Its downside is size: about **138 million parameters**, making it slow and memory-hungry.

VGG-16 has **13 conv + 3 dense** layers. VGG-19 has **16 conv + 3 dense**. (Only layers with weights are counted, not pooling layers.)

| Block | VGG-16 | VGG-19 | Output size (224 input) |
|---|---|---|---|
| 1 | 2 conv, 64 filters | 2 conv, 64 | 112×112×64 |
| 2 | 2 conv, 128 | 2 conv, 128 | 56×56×128 |
| 3 | 3 conv, 256 | 4 conv, 256 | 28×28×256 |
| 4 | 3 conv, 512 | 4 conv, 512 | 14×14×512 |
| 5 | 3 conv, 512 | 4 conv, 512 | 7×7×512 |
| End | Dense 4096, 4096, 1000 | same | 1000 |
| **Params** | **~138M** | **~144M** | |

(Each block ends with a 2×2 max pool that halves the size.)

**Where do 138 million parameters come from?** The first dense layer alone has $7 \times 7 \times 512 \times 4096 \approx 103$ million. All three dense layers together hold **about 89%** of the model. The 13 conv layers have only about 15 million, but they do most of the **computation**, because they run on large images.

Result: 2nd place in the 2014 ImageNet classification contest, behind GoogLeNet, which used about 20× fewer parameters.

⍰ **Interviewer may ask next**

- **"Why is VGG still used?"** Its features are simple and general, so it's handy for **transfer learning** and for "style transfer" (making a photo look like a painting).

---

### Q37. Why are stacked 3×3 convolutions better than one large 7×7 convolution?

**Simple answer**

Three 3×3 layers in a row **see the same 7×7 area** as one 7×7 layer, but use **about 45% fewer parameters** and include **three ReLUs instead of one**. So they're cheaper *and* can learn more complex patterns.

With $C$ channels going in and out (ignoring biases):

| Option | Area seen | Parameters | ReLUs |
|---|---|---|---|
| One 7×7 | 7×7 | $49C^2$ | 1 |
| Three 3×3 | 7×7 | $3 \times 9C^2 = 27C^2$ | 3 |
| One 5×5 | 5×5 | $25C^2$ | 1 |
| Two 3×3 | 5×5 | $18C^2$ | 2 |

For $C = 256$: one 7×7 layer has 3.2 million parameters, while three 3×3 layers have 1.8 million.

**⚠ Common mistake**

There's a trade-off. Three layers means **three sets of outputs to store** during training (more memory) and three steps in a row (more time). Fewer parameters doesn't automatically mean faster.

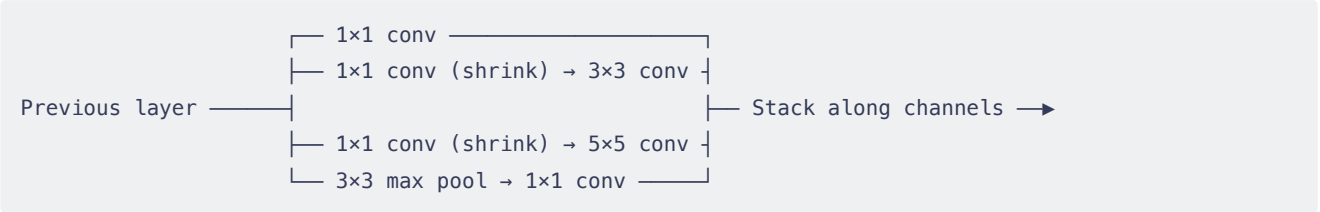
---

## 12. GoogLeNet / Inception

### Q38. What problem was the Inception module trying to solve?

**Simple answer**

Which filter size is best: 1×1, 3×3 or 5×5? Objects can be small or large, so the answer changes from image to image. Inception's idea: **don't choose. Run all of them side by side and stack the results together**, and let training decide which ones matter. The catch is that this is expensive, which Inception solves with 1×1 convolutions.




Every branch keeps the same width and height (using "same" padding), so the outputs can be stacked as channels. For example: 64 + 128 + 32 + 32 = **256** output channels.

---

## Q39. Show with numbers why the 1×1 convolution makes Inception affordable.

**Simple answer**

A 5×5 conv on an input with many channels is very expensive. If we first use a cheap 1×1 conv to **shrink the number of channels**, the 5×5 conv then has much less to do. This cuts the cost by about **10 times**. This shrinking step is called a "bottleneck".

**Setup:** input 28×28×192, and we want an output of 28×28×32 using a 5×5 conv.

**Option A, 5×5 directly:**

$$\underbrace{28 \times 28 \times 32}_{\text{output values}} \times \underbrace{5 \times 5 \times 192}_{\text{work per value}} \approx 120 \text{ million multiply-adds}$$

**Option B, 1×1 down to 16 channels first, then 5×5:**

$$\underbrace{28 \times 28 \times 16 \times 192}_{1\times1 \text{ step}} \approx 2.4\text{M} \; + \; \underbrace{28 \times 28 \times 32 \times 5 \times 5 \times 16}_{5\times5 \text{ step}} \approx 10.0\text{M} \; = \; \mathbf{12.4M}$$

About **10× less work** for the same output shape.

**⚠ Common mistake**

Doesn't squeezing 192 channels into 16 lose information? A little, but neighbouring channels usually carry a lot of **overlapping information**, so a learned compression keeps most of what matters. In practice accuracy stays good.

---

## Q40. What else was distinctive about GoogLeNet (Inception v1)?

**Simple answer**

GoogLeNet won ImageNet 2014 (**6.67% top-5 error**) with 22 layers but only about **5-7 million parameters**: around 12× fewer than AlexNet and 20× fewer than VGG. Besides Inception modules, it added **extra "helper" classifiers** during training, and replaced the big dense layers at the end with **Global Average Pooling**.

| Feature | Purpose |
|---|---|
| 9 Inception modules stacked | Looks at several scales at every stage |
| 1×1 bottlenecks | Keeps computation low |
| **Global Average Pooling** instead of big dense layers | Saves tens of millions of parameters and reduces overfitting |
| **Two helper (auxiliary) classifiers** in the middle | Give extra learning signal to middle layers during training; **removed at prediction time** |

Later versions: **Inception v2/v3** added BatchNorm and split big filters into smaller ones (a 5×5 becomes two 3×3s). **Inception-ResNet** added ResNet-style skip connections.

---

# 13. Residual Networks (ResNet)

## Q41. Why can't we just keep making a plain CNN deeper? What is the degradation problem?

**Simple answer**

When researchers made plain networks deeper than about 20 layers, the **56-layer network did worse than the 20-layer one, even on the training data**. That isn't overfitting (overfitting means good on training data and bad on new data). It's a **training difficulty**: the deeper network *could* in theory copy the shallower one and make its extra layers do nothing, but training couldn't find that solution.

```
             Training error      Test error
20 layers    lower  ✓            lower  ✓
56 layers    HIGHER ✗            HIGHER ✗     ← worse even on training data
```

This happened even with BatchNorm and good initialisation. The surprising cause: it's hard for a stack of layers to learn to **do nothing**, that is, to pass the input through unchanged.

**ResNet's idea:** make "do nothing" the easy default. Instead of asking a block to learn the full output, ask it to learn only the **change** (the "residual") to add to its input:

$$\text{output} = F(x) + x$$

If no change is needed, the block just learns $F(x) = 0$, which is easy (weights near zero). If a small improvement helps, it learns a small $F(x)$.

**An analogy:** editing a document instead of rewriting it from scratch. The original text is passed along, and each block only adds edits.

---

## Q42. What exactly does the skip connection do, for the forward pass and for gradients?

**Simple answer**

**Forward:** the input $x$ skips around the block and is added to its output, so each block only needs to learn a small **correction**. **Backward:** the gradient also flows straight back through that shortcut, without passing through any weights or activations. So it can't fade away along that path, and the early layers keep receiving a strong learning signal.

```
x ──────────────────────────────────────┐   (shortcut: x passes straight through)
│                                       │
└─► Conv 3×3 → BN → ReLU → Conv 3×3 → BN ─► (+) ─► ReLU ─► output
                    F(x)
```

The derivative of $x + F(x)$ with respect to $x$ is $1 + (\text{derivative of } F)$. That **"+1"** is the gradient's highway: even if $F$'s gradient is tiny, the total gradient doesn't vanish.

**⚠ Common mistake**

Saying "skip connections only fix vanishing gradients" is incomplete. The degradation problem appeared even with BatchNorm, which already helps with vanishing gradients. The deeper reason is that residuals make **"do nothing" the default**, which makes the network much easier to train. The lesson: *a network's design affects how easy it is to train, not just what it can represent.*

## Q43. What happens when the input and output of a residual block have different shapes?

**Simple answer**

You can only add two things of the same shape. When a block halves the image size and doubles the channels (for example 56×56×256 → 28×28×512), the shortcut uses a **1×1 conv with stride 2** to reshape $x$ so it matches. When the shapes already match, the shortcut is just a plain pass-through with no parameters.

$$\text{output} = F(x) + \text{Conv}_{1\times1,\ \text{stride}\ 2}(x)$$

## Q44. Walk me through ResNet-50. Why is it called "50", and what is a bottleneck block?

**Simple answer**

ResNet-50 uses **bottleneck blocks**: a 1×1 conv **shrinks** the channels, a 3×3 conv works on this thinner version, and another 1×1 conv **expands** the channels back. The four stages have **3, 4, 6 and 3** blocks. Count the layers with weights: 1 (first conv) + 16 blocks × 3 convs + 1 (final dense layer) = **50**. It has about **25.6 million parameters**: far fewer than VGG-16's 138 million, yet it's much more accurate.

**A bottleneck block** (first stage):

```
x (56×56×256)
 ├─▶ 1×1 conv, 64 filters    shrink:  256 → 64 channels
 │   3×3 conv, 64 filters    main work on just 64 channels (cheap)
 │   1×1 conv, 256 filters   expand:  64 → 256 channels
 └──────────────────────────▶ (+) → ReLU
```

How much it saves, for 256 channels:

| Block type | Weights |
|---|---|
| Bottleneck (1×1 → 3×3 → 1×1) | about **70,000** |
| Two plain 3×3 convs on 256 channels | about **1,180,000** |

**Full structure** (224×224 input):

| Stage | Output size | Blocks |
|---|---|---|
| First conv | 112×112 | 7×7 conv, 64 filters, stride 2 |
| Max pool | 56×56 | 3×3, stride 2 |
| Stage 1 | 56×56 | **3** bottleneck blocks (output 256 channels) |
| Stage 2 | 28×28 | **4** blocks (512 channels) |
| Stage 3 | 14×14 | **6** blocks (1024 channels) |
| Stage 4 | 7×7 | **3** blocks (2048 channels) |
| End | 1×1 | Global average pool → Dense 1000 → softmax |

The ResNet family:

| Model | Block type | Blocks per stage | Params |
|---|---|---|---|
| ResNet-18 | Basic (two 3×3) | 2, 2, 2, 2 | 11.7M |
| ResNet-34 | Basic | 3, 4, 6, 3 | 21.8M |
| ResNet-50 | Bottleneck | 3, 4, 6, 3 | 25.6M |
| ResNet-101 | Bottleneck | 3, 4, 23, 3 | 44.5M |
| ResNet-152 | Bottleneck | 3, 8, 36, 3 | 60.2M |

ResNet won ImageNet 2015 with a **3.57% top-5 error**, better than the commonly quoted human error of about 5%.

**Interviewer may ask next**

- **"Where does the image get smaller?"** In the first block of stages 2, 3 and 4, using stride 2, with a 1×1 conv on the shortcut so the shapes match.

---

# 14. Comparing CNN Architectures

## Q45. Compare LeNet, AlexNet, VGG, GoogLeNet and ResNet in one table.

| | LeNet-5 | AlexNet | VGG-16 | GoogLeNet | ResNet-50 |
|---|---|---|---|---|---|
| Year | 1998 | 2012 | 2014 | 2014 | 2015 |
| Layers (with weights) | ~7 | 8 | 16 | 22 | 50 |
| Parameters | ~60K | ~60M | ~138M | ~6–7M | ~25.6M |
| ImageNet top-5 error | – (digits) | 15.3% | ~7.3% | 6.67% | ~5.3% (3.57% for the ResNet ensemble) |
| Main idea | Conv + pool recipe | Scale it up: ReLU, dropout, GPUs, augmentation | Go deep with only 3×3 filters | Several filter sizes in parallel + 1×1 shrinking + GAP | Skip connections + bottlenecks |
| Main weakness | Very small scale | Big filters, huge dense layers | Huge and slow | Complicated design | (Became the standard baseline) |

## Q46. Tell the story of LeNet → ResNet in under a minute.

**Simple answer**

Each new design fixed the weakness of the one before it.

```
LeNet (1998)      → showed conv + pool + backprop can learn to see.
                    Weakness: tiny; not enough data or compute.
AlexNet (2012)    → scaled it up: big data + GPUs + ReLU + dropout.
                    Weakness: huge dense layers, ad-hoc design.
VGG (2014)        → went deeper with a simple design: only 3×3 filters.
                    Weakness: 138M parameters, slow.
GoogLeNet (2014) → several filter sizes in parallel, cheaply, using 1×1 convs.
                    Weakness: still hard to go very deep; complex.
ResNet (2015)     → skip connections made 50–152+ layers trainable.
                    → became the standard backbone for most vision tasks.
```

> **"The weakness of each architecture became the problem the next one solved."** That's a great sentence to say out loud in an interview.

---

# 15. Transfer Learning and Data Augmentation

## Q47. What is transfer learning? When do you freeze layers vs fine-tune?

**Simple answer**

Instead of training from scratch, take a network **already trained on a huge dataset** (usually ImageNet), replace its last layer with one for your own classes, and train on your data. The early layers have learned **general** things (edges, textures) that are useful for almost any image task. The later layers are more **specific** to the original task. How much you retrain depends on **how much data you have** and **how similar it is** to the original images.

**An analogy:** someone who already knows how to drive a car learns to drive a truck much faster than someone who has never driven. They reuse most of what they know.

| | Your images look like ImageNet photos | Your images are very different (X-rays, satellite) |
|---|---|---|
| **Small dataset** | Freeze the pretrained part; train only the new last layer | Freeze the early layers; carefully retrain the later ones |
| **Large dataset** | Fine-tune the whole network with a small learning rate | Fine-tune everything (pretrained weights are still a better start than random) |

("Freeze" means those weights won't change during training.)

**The usual two-step recipe (Keras):**

```
base = keras.applications.ResNet50(include_top=False, weights="imagenet",
                                   input_shape=(224, 224, 3))
base.trainable = False                                    # Step 1: freeze the pretrained part

inputs = keras.Input(shape=(224, 224, 3))
x = keras.applications.resnet50.preprocess_input(inputs)  # same preprocessing as the original
training
x = base(x, training=False)                                # keep BatchNorm in prediction mode
x = layers.GlobalAveragePooling2D()(x)
x = layers.Dropout(0.3)(x)
outputs = layers.Dense(num_classes, activation="softmax")(x)
model = keras.Model(inputs, outputs)
model.compile(keras.optimizers.Adam(1e-3), "sparse_categorical_crossentropy", ["accuracy"])
model.fit(train_ds, validation_data=val_ds, epochs=10)

base.trainable = True                                     # Step 2: fine-tune the top layers
for layer in base.layers[:-30]:
    layer.trainable = False                               # keep the early layers frozen
model.compile(keras.optimizers.Adam(1e-5), "sparse_categorical_crossentropy", ["accuracy"])
model.fit(train_ds, validation_data=val_ds, epochs=10)
```

⚠ **Common mistake**

- **Train the new last layer first.** If you unfreeze everything right away, the new random layer sends large, messy gradients back and damages the pretrained weights.
- **Use a much smaller learning rate when fine-tuning** (10–100× smaller).
- **In Keras, compile again after changing** `trainable`, or the change won't take effect.
- **Use the same preprocessing** the pretrained model was trained with (`preprocess_input`). Feeding differently scaled pixels quietly hurts accuracy.

---

## Q48. What data augmentation would you use for images, and when can augmentation hurt?

🔥 **Simple answer**

Data augmentation creates new training examples by **randomly changing existing images**: flipping, cropping, rotating slightly, changing brightness, and so on. The model learns that these changes don't affect the answer, so it generalises better. The key rule: **the change must not change the correct label**. A bad augmentation quietly teaches the model wrong things.

| Type | Examples |
|---|---|
| Position and shape | Random crop, horizontal flip, small rotation, zoom, shift |
| Colour and lighting | Brightness, contrast, saturation, blur, noise |
| Hiding parts | Cutout (cover a random square) |
| Mixing images | **Mixup** (blend two images and their labels), **CutMix** (paste a patch from another image) |

**When augmentation hurts:**

| Task | Bad augmentation | Why |
|---|---|---|
| Digit recognition | Rotating 180° | A 6 becomes a 9 |
| Reading text | Horizontal flip | Letters become mirrored |
| Chest X-rays | Horizontal flip | The heart is on the left; a flipped X-ray isn't a real body |
| Traffic signs | Horizontal flip | "Turn left" becomes "turn right" |
| Fruit ripeness | Colour changes | The colour *is* the answer |
| Object detection | Moving or flipping the image without moving the boxes | The boxes no longer match the objects |

```
augment = keras.Sequential([
    layers.RandomFlip("horizontal"),
    layers.RandomRotation(0.05),
    layers.RandomZoom(0.1),
    layers.RandomContrast(0.2),
])
# These layers only change images during training, not during prediction.
```

⚠ **Common mistake**

Augment **only the training data**. Validation and test data should look like the real images the model will see after deployment.

---

# 16. Classification vs Detection vs Segmentation

## Q49. What's the difference between classification, localization, detection, and segmentation?

**Simple answer**

Each one answers a more detailed question. **Classification:** what is in the image? **Localization:** what is the main object, and where is it? **Detection:** where is *every* object, with a box around each? **Segmentation:** exactly which **pixels** belong to what? The more detail you want, the harder the model and the more expensive the labelling.

| Task | Question | Output | Example models | How it's scored |
|---|---|---|---|---|
| Classification | What's in the image? | One label (with probabilities) | ResNet, EfficientNet | Accuracy |
| Classification + localization | What's the main object, and where? | Label + 1 box | CNN with two outputs | Accuracy + IoU |
| Object detection | What are all the objects, and where? | Many boxes, each with a label | YOLO, Faster R-CNN | mAP |
| Semantic segmentation | What is each pixel? | A label for every pixel | U-Net, DeepLab | mIoU, Dice |
| Instance segmentation | Which object does each pixel belong to? | A separate mask for each object | Mask R-CNN | Mask mAP |

**Labelling cost grows too:** a label takes seconds, a box about 10 seconds, and an exact outline of an object can take minutes. That cost is a real factor when choosing the task.

**Interviewer may ask next**

- **"A factory wants to find defective parts. Which task?"** It depends on what decision they need. Just pass/fail → **classification**. A worker needs to know where the scratch is → **detection**. They need to measure the scratch's size → **segmentation**. **Pick the task that fits the decision, not the fanciest model.**

---

## Q50. Semantic vs instance vs panoptic segmentation: what's the difference?

**Simple answer**

**Semantic** gives every pixel a class but doesn't separate objects: three cars side by side become one "car" area. **Instance** gives each object its own mask (car 1, car 2, car 3) but usually ignores the background. **Panoptic** does both: every pixel gets a class, and every individual object gets its own ID.

```
Scene: three cars on a road
```

```
Semantic:    [road road car  car  car  car  car  road]     knows "car", not which car
Instance:    [ -    -   car1 car1 car2 car3 car3  -  ]     separate cars, ignores road
Panoptic:    [road road car1 car1 car2 car3 car3 road]     both
```

Uses: counting cells or people → instance. Finding the drivable road → semantic. Full scene understanding for self-driving cars → panoptic.

---

## Q51. How does a network produce a pixel-level output? Explain FCN and U-Net.

> 🔥 **Simple answer**
>
> A classifier shrinks the image down to a single answer. Segmentation needs an answer for **every pixel**, so the output must be full size again. We use an **encoder-decoder** design. The encoder shrinks the image and figures out **what** is there. The decoder enlarges it back to full size to work out **where**. **U-Net** also adds **skip connections** that pass sharp, detailed features from the encoder directly to the decoder, so the edges stay precise.

**FCN (Fully Convolutional Network, 2015):** take a classification CNN, remove its dense layers so it outputs a small "map" of class scores, then enlarge that map back to full image size.

**U-Net (2015, designed for medical images):**

```
Encoder (shrinking)                          Decoder (growing back)
full size ── conv,conv ──────── copy ──────► join ─ conv,conv ─► pixel-by-pixel mask
   ↓ shrink                                     ↑ enlarge
 conv,conv ──────────────────── copy ──────► join ─ conv,conv
   ↓ shrink                                     ↑ enlarge
 conv,conv ──────────────────── copy ──────► join ─ conv,conv
   ↓ shrink                                     ↑ enlarge
         ──────────── bottom (smallest image, most "meaning") ──────────
```

Why the skip connections matter:

```
Deep layers   → know WHAT it is ("tumour"), but blurry about WHERE
Early layers  → know exactly WHERE the edges are, but not WHAT
Skip connections give the decoder both.
```

U-Net works well even with **small datasets** (a few hundred images), which is why it's so popular in medical imaging.

**Dice score**, a common segmentation measure, is useful when the object is tiny (for example a tumour covering 1% of the image):

$$\text{Dice} = \frac{2 \times \text{overlap}}{\text{predicted area} + \text{true area}}$$

> ⚠ **Common mistake**
>
> U-Net's skip connections are **not** the same as ResNet's. ResNet **adds** the input to a block's output, to make training easier. U-Net **joins (concatenates)** encoder features onto the decoder, to bring back fine detail.

---

### Q52. What is a transposed convolution? What's the checkerboard artifact?

**Simple answer**

A transposed convolution is a **learnable way to make an image bigger**. It's the opposite of a strided conv. With stride 2, it doubles the width and height. If the filter size doesn't divide evenly by the stride, some output pixels get more "paint" than others, which creates a **checkerboard pattern**.

Output size: $O = (W - 1) \times S + K$. Example: $W = 4$, $S = 2$, $K = 2$ → output 8.

**The fix:** use a filter size divisible by the stride, or simply resize the image (`UpSampling2D`) and then apply a normal conv. That's often cleaner.

```
layers.Conv2DTranspose(64, kernel_size=2, strides=2, padding="same")   # learnable ×2
layers.UpSampling2D(2); layers.Conv2D(64, 3, padding="same")          # simple ×2 + conv
```

---

## 17. Object Detection and YOLO

### Q53. Why is object detection harder than classification?

**Simple answer**

Detection must find an **unknown number** of objects. For each one it must give a **label** (what) and a **box** (where), handle objects of **very different sizes**, and remove **duplicate** guesses. Classification only gives one answer per image.

| Challenge | Why it's hard | Usual solution |
|---|---|---|
| Unknown number of objects | A network always outputs a fixed-size result | Predict many candidates, keep the confident ones |
| Exact location | Box coordinates are continuous numbers | Predict boxes relative to grid cells or anchors |
| Different sizes | A person can be 20 or 600 pixels tall | Predict at several image scales |
| Duplicates | Nearby candidates all fire on the same object | Non-Maximum Suppression (NMS) |
| Mostly background | Thousands of empty spots for each real object | Objectness scores, focal loss |
| Scoring | Both the label and the location must be right | IoU + mAP |

---

### Q54. Two-stage vs one-stage detectors: what's the trade-off?

**Simple answer**

**Two-stage** detectors (the R-CNN family) first **suggest regions** that might contain objects, then **check each region** carefully. They are accurate but slower. **One-stage** detectors (YOLO, SSD) predict all boxes and labels **in one go**. They are much faster, and today they're about as accurate.

| | Two-stage (Faster R-CNN) | One-stage (YOLO, SSD) |
|---|---|---|
| How it works | Propose regions → classify and refine each one | Predict everything in a single pass |

| | Two-stage (Faster R-CNN) | One-stage (YOLO, SSD) |
|---|---|---|
| Speed | Slower (about 5–20 images/sec, classic versions) | Fast (30–150+ images/sec) |
| Accuracy | Historically higher | Gap now mostly closed |
| Best for | When accuracy matters most | Real-time: video, robots, phones |

The R-CNN family in one line: **R-CNN** (very slow: ran the CNN about 2,000 times per image) → **Fast R-CNN** (runs the CNN once and shares it) → **Faster R-CNN** (also learns *where* to look).

**Interviewer may ask next**

- **"Why were one-stage detectors less accurate at first?"** They check tens of thousands of positions, and almost all are empty background. These easy background examples dominated training. **Focal loss** (2017) fixed this by giving less weight to easy examples.

---

## Q55. How does YOLO (v1) work? What exactly is in its output?

**Simple answer**

YOLO ("You Only Look Once", 2016) splits the image into a **7×7 grid**. The grid cell containing an object's **centre** is responsible for detecting it. Each cell predicts **2 boxes** (each with position, size and a confidence score) plus **20 class probabilities**. That gives an output of **7 × 7 × 30**, all in one pass through the network.

$$\text{Output} = S \times S \times (B \times 5 + C) = 7 \times 7 \times (2 \times 5 + 20) = 7 \times 7 \times 30$$

where $S$ = grid size (7), $B$ = boxes per cell (2), $C$ = number of classes (20), and 5 = (x, y, w, h, confidence).

The 30 numbers for each cell:

```
[ x1, y1, w1, h1, conf1,   x2, y2, w2, h2, conf2,   p(class 1), …, p(class 20) ]
  └──────── box 1 ────────┘ └──────── box 2 ────────┘ └──── shared by both boxes ────┘
```

| Value | Meaning |
|---|---|
| $x, y$ | Centre of the box, measured inside its grid cell (0 to 1) |
| $w, h$ | Width and height, as a fraction of the whole image (0 to 1) |
| confidence | "How sure am I there's an object here, and how well does my box fit it?" |
| class probabilities | "If there's an object in this cell, which class is it?" |

At prediction time: final score = class probability × confidence. Keep the high scores, then remove duplicates with NMS.

**Training details worth knowing:**

- Box position errors are given **5× more weight**, because location matters.
- Empty cells get **0.5× weight**, because most cells are empty and would otherwise dominate training.
- It predicts the **square root** of width and height, so a 10-pixel mistake counts more on a small box than on a big one.

**Speed:** about **45 images per second**, which is real time. It was less accurate than Faster R-CNN at the time. Because YOLO sees the **whole image at once**, it makes fewer mistakes on background.

---

## Q56. What are YOLOv1's limitations, and how did later versions address them?

**Simple answer**

Each grid cell can only predict 2 boxes and **one class**, so YOLOv1 struggles with **small objects close together** (like a flock of birds) and unusual shapes. The coarse 7×7 grid also limits how precisely it can place boxes. Later versions added **anchor boxes**, **predictions at several scales**, and better backbones.

| Version | Main improvements |
|---|---|
| YOLOv2 (2017) | BatchNorm; **anchor boxes** picked automatically by clustering the training boxes; finer 13×13 grid |
| YOLOv3 (2018) | Deeper network with skip connections; predicts at **3 different scales**, much better for small objects |
| YOLOv4/v5 (2020) | Better augmentation (e.g. "mosaic": 4 images combined into 1), better box loss |
| YOLOv8 and later | **No anchors needed**; can also do segmentation and pose |

A modern YOLO has three parts: **backbone** (extracts features) → **neck** (combines features from different scales) → **head** (predicts boxes and classes).

---

## Q57. What are anchor boxes and why were they introduced?

**Simple answer**

Anchors are **pre-set box shapes** (tall, wide, square, small, large) placed at each grid position. Instead of predicting a box from nothing, the network predicts **small adjustments** to the closest anchor. This is easier to learn, and it lets one position detect several differently shaped objects, such as a tall person and a wide car in the same spot.

**An analogy:** it's easier to say "take this standard frame and stretch it a bit" than to draw a frame from scratch every time.

When training, each real box is matched to the anchor it overlaps most, and that anchor learns to predict it.

**Interviewer may ask next**

- **"Why have newer detectors dropped anchors?"** Anchors add extra settings (sizes, shapes, matching rules) that need tuning for each dataset. Anchor-free detectors (like YOLOv8) predict boxes directly and are simpler, with similar accuracy.

---

## Q58. What is IoU? Compute one.

**Simple answer**

IoU (Intersection over Union) measures **how well two boxes overlap**: the shared area divided by the total area covered by both. 1 means a perfect match, 0 means no overlap. A detection usually counts as correct if its IoU with the real box is **at least 0.5**.

$$\text{IoU} = \frac{\text{overlap area}}{\text{area of A} + \text{area of B} - \text{overlap area}}$$

(We subtract the overlap once so it isn't counted twice.)

**Worked example.** Boxes are given as (left, top, right, bottom):

- Real box A = (0, 0, 10, 10) → area 100
- Predicted box B = (5, 5, 15, 15) → area 100
- Overlap = (5, 5, 10, 10) → 5 × 5 = 25
- Union = 100 + 100 − 25 = 175
- **IoU = 25 / 175 ≈ 0.14.** That's a miss at the 0.5 threshold, even though the boxes clearly overlap.

```
def iou(a, b):
    left, top = max(a[0], b[0]), max(a[1], b[1])
    right, bottom = min(a[2], b[2]), min(a[3], b[3])
    overlap = max(0, right - left) * max(0, bottom - top)     # 0 if they don't touch
    area_a = (a[2] - a[0]) * (a[3] - a[1])
    area_b = (b[2] - b[0]) * (b[3] - b[1])
    return overlap / (area_a + area_b - overlap)
```

**Interviewer may ask next**

- **"Can IoU be used as a training loss?"** Yes ($1 - \text{IoU}$), but when boxes don't overlap at all, IoU is 0 no matter how far apart they are, so there's no signal to improve. Improved versions (GIoU, DIoU, CIoU) add a term for distance so the model still learns.

---

## Q59. Why do detectors produce duplicate boxes, and how does Non-Maximum Suppression work?

**Simple answer**

Several nearby grid cells or anchors all "see" the same object, so you get several boxes around it. **NMS** cleans this up: keep the most confident box, delete other boxes of the same class that overlap it heavily (IoU above about 0.5), and repeat with what's left.

**Steps:**

1. Sort all boxes by confidence, highest first.
2. Keep the top box.
3. Delete any remaining box that overlaps it by more than the threshold.
4. Repeat with the next box still in the list.

```
def nms(boxes, scores, iou_thresh=0.5):
    order = scores.argsort()[::-1]          # most confident first
    keep = []
    while len(order) > 0:
        best = order[0]
        keep.append(best)
        rest = order[1:]
        order = rest[[iou(boxes[best], boxes[i]) < iou_thresh for i in rest]]
    return keep
```

This is usually done **separately for each class**, so a "person" box doesn't delete an overlapping "bicycle" box.

| Threshold | Effect |
|---|---|
| Too low (0.3) | Deletes real objects that happen to overlap, e.g. in crowds |

| Threshold | Effect |
|---|---|
| Too high (0.8) | Leaves duplicate boxes behind |

**Interviewer may ask next**

- **"NMS struggles with crowds. Alternatives?" Soft-NMS** lowers the scores of overlapping boxes instead of deleting them outright.

---

## Q60. How is a detector evaluated? Explain mAP.

**Simple answer**

For each class: sort all predictions by confidence and mark each as **correct** (IoU ≥ 0.5 with a real box not already matched) or **wrong**. Then calculate precision and recall at every confidence level and draw a curve. **AP** (Average Precision) is the area under that curve. **mAP** is the **average AP across all classes**. The COCO benchmark is stricter: it averages over IoU thresholds from 0.5 to 0.95, which rewards very precise boxes.

Quick reminder:

- **Precision:** of the boxes I predicted, how many were right?
- **Recall:** of the real objects, how many did I find?

```
For class "dog", sorted by confidence:
confidence  0.95  0.91  0.85  0.80  0.70 …
result      ✓     ✓     ✗     ✓     ✗    …
→ precision and recall at each step → curve → AP for "dog"
mAP = average of AP(dog), AP(cat), AP(car), …
```

**Why not accuracy?** In detection there's no fixed list of "negatives": any spot in the image could be a wrong box. Precision and recall still make sense; accuracy doesn't.

**Common mistake**

Each real object can only be matched **once**. A second box on the same object counts as **wrong**. That's exactly why NMS improves mAP.

---

# 18. Why Sequence Models

## Q61. CNNs handle images. What changes when the data is a sequence, and why doesn't a normal network work?

**Simple answer**

In a sequence, **order matters** and the **length varies**. "Dog bites man" means something very different from "man bites dog". A normal (feed-forward) network takes a fixed-size input, has **no memory** of earlier inputs, and would need separate weights for every position. We need a model that reads **one step at a time**, uses the **same weights at every step**, and **remembers** what it has seen so far.

Examples of sequence data: text, speech, sensor readings, stock prices, heartbeat signals, video frames.

| Need | Normal network | RNN |
|---|---|---|
| Handle inputs of any length | × | ✓ |
| Recognise a pattern at any position | × | ✓ (same weights at every step) |
| Remember earlier inputs | × | ✓ (hidden state = memory) |

Common sequence task shapes:

| Shape | Example |
|---|---|
| One-to-many | Image → caption (sequence of words) |
| Many-to-one | Review → positive/negative |
| Many-to-many (same length) | Tag each word in a sentence (noun, verb, ...) |
| Many-to-many (different lengths) | Translation: English sentence → French sentence |

**Interviewer may ask next**

- **"Can a CNN handle sequences?"** Yes, using 1D convolutions along time. They're fast, but they only see a limited window. An RNN's memory can, in principle, carry information much further.

---

# 19. Recurrent Neural Networks (RNN)

## Q62. What is the basic idea behind an RNN? Give the equations and shapes.

**Simple answer**

An RNN reads a sequence **one step at a time**. At each step it combines the **current input** with its **memory of everything before** (the hidden state) to make a new memory. The **same weights are used at every step**. That's why it works for any length.

**An analogy:** reading a book one word at a time. After each word you update your understanding of the story, using both the new word and what you already understood.

$$h_t = \tanh(W_x\, x_t + W_h\, h_{t-1} + b)$$

where:

- $x_t$ = input at step $t$ (e.g. the current word)
- $h_{t-1}$ = memory from the previous step
- $h_t$ = updated memory
- $W_x, W_h, b$ = weights, **shared across all steps**

```
        out1        out2        out3
         ↑           ↑           ↑
h0 ──► [RNN] ─h1─► [RNN] ─h2─► [RNN] ─h3─► …      (the SAME cell is reused at each step)
         ↑           ↑           ↑
        x1          x2          x3
```

**Parameter count** (input size $d$, hidden size $h$): $h \times d + h \times h + h$. For $d = 100$, $h = 64$: **10,560** parameters, whether the sequence has 5 steps or 5,000.

```
layers.SimpleRNN(64)                          # many-to-one: returns only the final memory
layers.SimpleRNN(64, return_sequences=True)   # many-to-many: returns the memory at every step
```

⚠ **Common mistake**

When stacking RNN layers, every layer except the last needs `return_sequences=True`. The next layer needs a whole sequence as input, not just the final memory. Forgetting this causes a shape error in Keras.

---

## Q63. What is Backpropagation Through Time, and why do basic RNNs struggle with long sequences?

**Simple answer**

To train an RNN, we "unroll" it across all time steps, which makes it look like a very deep network, and run normal backprop. That's **Backpropagation Through Time (BPTT)**. The gradient from a late step back to an early step is **multiplied once for every step in between**. Over many steps it usually **shrinks to almost zero**, so the RNN can't learn that something from 50 words ago matters. Sometimes it **explodes** instead.

It's the same vanishing/exploding gradient problem as in deep networks (Q14), but now the "depth" is the **length of the sequence**.

**An example that needs long memory:**

```
"The keys to the old wooden cabinet in the hallway ___ on the table."
     ↑                                                    ↑
  "keys" is plural                        so we need "are", not "is" (9 words later)
```

A basic RNN usually only remembers useful information for about 10 steps.

| Problem | Fix |
|---|---|
| Exploding gradients | **Gradient clipping**: cap the gradient size |
| Vanishing gradients | Use **LSTM or GRU** |
| Very long sequences are slow to train | **Truncated BPTT**: only backprop through the last $k$ steps |

⚠ **Common mistake**

Gradient clipping fixes **exploding** gradients but **not** vanishing ones. You can't scale up a gradient that is already basically zero. Vanishing needs a different design, which is why LSTM was invented.

---

# 20. LSTM and GRU

## Q64. What problem does LSTM solve, and how? Give the equations.

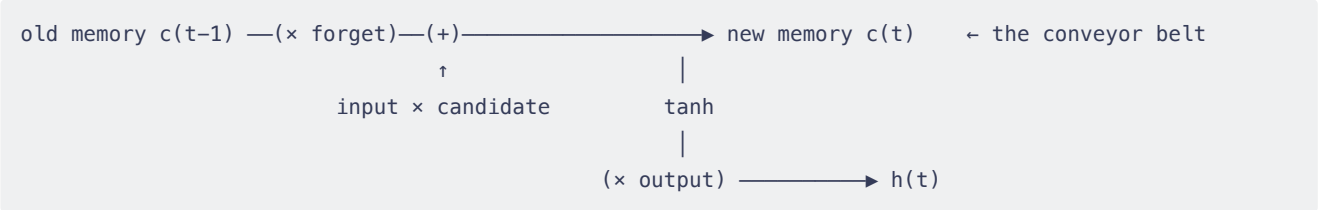
**Simple answer**

LSTM (1997) gives the RNN a separate **long-term memory**, called the **cell state**, like a conveyor belt running through the whole sequence. Three **gates** control it: the **forget gate** decides what to erase, the **input gate** decides what new information to write, and the **output gate** decides what to reveal right now. The memory is updated by **adding**, not by repeated multiplying, so information and gradients can survive for hundreds of steps.

**An analogy:** the cell state is a notebook you carry through a long lecture. At each moment you can cross out old notes (forget gate), write new notes (input gate), and decide which notes to read out loud right now (output gate).

$$
\begin{aligned}
f_t &= \sigma(W_f[h_{t-1}, x_t] + b_f) && \text{forget gate: what to erase } (0 = \text{erase}, 1 = \text{keep}) \\
i_t &= \sigma(W_i[h_{t-1}, x_t] + b_i) && \text{input gate: how much new info to write} \\
\tilde{c}_t &= \tanh(W_c[h_{t-1}, x_t] + b_c) && \text{candidate: the new info itself} \\
c_t &= f_t \odot c_{t-1} + i_t \odot \tilde{c}_t && \text{update memory: keep some old + add some new} \\
o_t &= \sigma(W_o[h_{t-1}, x_t] + b_o) && \text{output gate: what to reveal} \\
h_t &= o_t \odot \tanh(c_t) && \text{output for this step}
\end{aligned}
$$

($\sigma$ = sigmoid, giving values between 0 and 1. $\odot$ = multiply element by element. $[h_{t-1}, x_t]$ = previous output and current input joined together.)




**Why gradients survive:** the memory update is $c_t = f_t \times c_{t-1} + \ldots$. Going backwards, the gradient along the conveyor belt is just multiplied by the **forget gate** $f_t$. If the gate stays close to 1 ("keep"), the gradient passes through almost unchanged, even across many steps. In a basic RNN, it is instead multiplied by weights and tanh slopes at every step, and it fades.

**⚠ Common mistake**

LSTMs **reduce** the vanishing gradient problem; they don't eliminate it. In practice, their memory still lasts hundreds of steps, not thousands. That's one reason attention, and later Transformers, took over.

---

## Q65. Explain each LSTM gate with an intuitive example.

**Simple answer**

Each gate is a number between 0 and 1 (from a sigmoid), acting like a **dimmer switch**: 0 = block completely, 1 = let everything through. **Forget** chooses what old memory to keep. **Input** chooses how much new information to store. **Output** chooses what part of the memory is useful right now.

Running example, predicting the next word: *"Alice went to the shop. Later, Bob arrived and **he** ..."*

| Part | Job | Range | In the example |
|---|---|---|---|
| Forget gate | Keep or erase old memory | 0 to 1 | When "Bob" appears, erase the stored info about "Alice" as the subject |

| Part | Job | Range | In the example |
|---|---|---|---|
| Input gate | How much new info to store | 0 to 1 | Strongly store "subject = Bob, one person" |
| Candidate | The new info itself | –1 to 1 | Information about "Bob" |
| Output gate | What to reveal right now | 0 to 1 | Before predicting the verb, reveal "the subject is one person" |

**Why sigmoid for gates but tanh for the candidate?** Gates answer "**how much**?", so they need a value from 0 to 1. The candidate is the **content** itself. It needs to be able to push memory values both up and down, so it uses –1 to 1.

**Why have an output gate at all?** The memory might hold things that matter **later** but not **now**. The output gate lets the LSTM keep information quietly without it affecting the current prediction.

---

## Q66. How many parameters does an LSTM layer have?

**Simple answer**

An LSTM has **4 copies** of a basic RNN's weights: one each for the forget gate, input gate, output gate and candidate. So it has 4 × (RNN parameters) $= 4 \times [h(h+d)+h]$.

**Example:** input size $d = 100$, hidden size $h = 128$:

$$4 \times [128 \times (128+100)+128] = 4 \times 29{,}312 = \mathbf{117{,}248}$$

```
model = keras.Sequential([keras.Input(shape=(None, 100)), layers.LSTM(128)])
model.summary()   # LSTM params: 117,248
```

| Layer ($d = 100$, $h = 128$) | Formula | Parameters |
|---|---|---|
| SimpleRNN | $h(h+d)+h$ | 29,312 |
| GRU | about $3\times$ RNN | ~88K (Keras shows 88,320) |
| LSTM | $4\times$ RNN | 117,248 |

The count does **not** depend on the sequence length.

---

## Q67. What is a GRU, and how does it compare to LSTM?

**Simple answer**

A GRU (2014) is a **simpler LSTM**. It combines the memory and the output into one state, and uses **two gates instead of three**. The **update gate** decides how much to keep the old memory versus replace it with new information. The **reset gate** decides how much of the past to use when creating that new information. It has about 25% fewer parameters, trains faster, and often works just as well.

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

In words: the new memory is a **blend** of the old memory and the new candidate. The update gate $z_t$ controls the mix (0 = keep everything old, 1 = replace with new).

| | LSTM | GRU |
|---|---|---|
| Gates | 3 (forget, input, output) | 2 (update, reset) |

| | LSTM | GRU |
|---|---|---|
| Memory | Separate cell state + output | One combined state |
| Parameters | 4× RNN | 3× RNN |
| Choose when | Long sequences, lots of data | Smaller datasets, faster training |

Neither one always wins. Try both if it matters.

---

## Q68. What is a bidirectional RNN, and when can't you use one?

**🔥 Simple answer**

A bidirectional RNN runs **two RNNs**: one reads the sequence forwards and one reads it backwards. Their outputs are combined, so every position knows about **both what came before and what comes after**. That's great when you have the whole sequence up front (for example classifying a full sentence). You **can't** use it when you must predict before the future arrives: live speech, forecasting, or generating text word by word.

```
"He said Teddy ___"   →  "Teddy bears are on sale"          (a toy)
                      →  "Teddy Roosevelt was a president"  (a person)
The words AFTER "Teddy" decide what it means, and the backward RNN sees them.
```

```
layers.Bidirectional(layers.LSTM(64))   # output size 128 (64 forwards + 64 backwards)
```

**⚠ Common mistake**

Using a bidirectional model for **forecasting** (like tomorrow's sales) is **cheating**. It sees future values while predicting the present. It will look amazing in testing and fail in real use.

---

# 21. Sequence-to-Sequence (Seq2Seq)

## Q69. Why do we need Seq2Seq models? How does the encoder-decoder architecture work?

**🔥 Simple answer**

In tasks like translation, both the input and the output are sequences, but they have **different lengths** and the words don't line up one to one ("I am learning" → "J'apprends"). Seq2Seq (2014) uses two RNNs. The **encoder** reads the whole input and squeezes it into a summary vector (the **context vector**). The **decoder** starts from that summary and writes the output **one word at a time**, feeding each word back in, until it produces an "end" token.

**An analogy:** a human interpreter listens to the whole sentence (encoder), holds the meaning in their head (context vector), then speaks it in the other language word by word (decoder).

```
Encoder                                          Decoder
 "I"  "am"  "learning"                           <START>    "J'"     "apprends"
  ↓     ↓       ↓                                  ↓         ↓          ↓
[RNN]→[RNN]→[RNN] ─── context vector ──────→ [RNN] → [RNN] → [RNN]
                                                   ↓         ↓          ↓
                                                 "J'"   "apprends"    <END>
```

Used for: translation, summarisation, chatbots, speech-to-text, and multi-step forecasting.

---

## Q70. What is teacher forcing? What is exposure bias?

**Simple answer**

During training, instead of feeding the decoder **its own** previous guess, we feed it the **correct** previous word. That's **teacher forcing**. It makes training faster and more stable, because one early mistake doesn't mess up the rest of the sentence. The downside, called **exposure bias**, is that during real use the model must rely on its own guesses, something it never practised, so mistakes can snowball.

```
Correct sentence:    <START> Le  chat  est  noir
Training input:      <START> Le  chat  est           ← the correct words (teacher forcing)
Real-use input:      <START> Le  chien est           ← the model's own guesses (may be wrong)
```

**An analogy:** learning to ride a bike with training wheels. Easier to learn, but the first ride without them is wobbly.

**A fix: scheduled sampling**. During training, gradually replace the correct words with the model's own guesses.

---

## Q71. Greedy decoding vs beam search?

**Simple answer**

**Greedy** decoding picks the single most likely word at each step. It's fast, but an early choice that looks best can lead to a worse sentence overall. **Beam search** keeps the **top few** (say 5) partial sentences at each step, extends each one, and at the end picks the best complete sentence.

```
Step 1:  "The" (0.5)   "A" (0.4)          greedy picks "The"
Step 2:  "The dog" = 0.5 × 0.3 = 0.15
         "A dog"   = 0.4 × 0.9 = 0.36      ← actually better overall
         Beam search (keeping 2) finds "A dog"; greedy misses it.
```

Practical detail: without a correction, beam search tends to prefer **short** sentences, because every extra word multiplies in another number below 1. So scores are adjusted for length.

**Interviewer may ask next**

- **"Is beam search always best?"** For translation, yes. For creative text such as stories or chat, it gives boring, repetitive output, so we use **random sampling** methods (top-k, top-p) instead.

---

## Q72. Why is a single fixed context vector a problem?

**Simple answer**

The encoder must squeeze the **entire input**, whether 5 words or 50, into **one fixed-size vector**, and the decoder sees nothing else. For long sentences there's too much to fit, so details get lost and translation quality drops sharply. This **bottleneck** is exactly what attention was invented to fix.

```
word 1  ─┐
word 2  ─┤
  …      ├──► [ one vector of, say, 512 numbers ] ──► decoder must rebuild everything from it
word 50 ─┘
```

**An analogy:** summarising a whole chapter on a sticky note, then translating the chapter using only that sticky note.

---

## 22. Attention Mechanism

### Q73. What is the key idea of attention? Walk through the computation step by step.

> **Simple answer**
>
> Instead of squeezing the input into one vector, **keep the encoder's output for every input word**. Then, each time the decoder writes a word, it **looks back at all the input words** and decides which ones matter most **right now**. It builds a fresh summary weighted towards those words. It's like a translator glancing back at the right part of the original sentence for each word they write.

**At each decoder step:**

1. **Score:** how relevant is each input word to what I'm about to write?
2. **Softmax:** turn the scores into weights that add up to 1.
3. **Context:** make a weighted average of the input words' encoder outputs.
4. **Write:** use this context (plus the decoder's own memory) to produce the next word.

$$\text{weights } \alpha_i = \text{softmax}(\text{score}_i), \qquad \text{context} = \sum_i \alpha_i \, h_i$$

where $h_i$ is the encoder output for input word $i$.

```
Encoder outputs:   h1("I")  h2("am")  h3("learning")  h4("French")
Writing "français":
attention weights:  0.02     0.03       0.10            0.85
context = mostly h4 ("French")   ← the decoder "looks at" the right word
```

Because the context is rebuilt at every step, the bottleneck is gone. The decoder can reach any input word directly, no matter how far back it is.

**Bonus:** the attention weights can be drawn as a grid, showing which input words the model looked at for each output word. That's a nice built-in explanation.

> **Interviewer may ask next**
>
> - **"Does attention also help with gradients?"** Yes. It creates **direct shortcuts** from every output to every input word, so gradients don't have to travel through dozens of RNN steps.

---

## Q74. Bahdanau vs Luong attention: what's the difference?

**Simple answer**

They differ mainly in **how they calculate the relevance score**. **Bahdanau** (2014) uses a small neural network to compare the decoder state with each encoder output ("additive" attention). **Luong** (2015) uses a simpler **dot product**, basically multiplying the two vectors to measure similarity ("multiplicative" attention). Luong's is simpler and faster.

| | Bahdanau (additive) | Luong (multiplicative) |
|---|---|---|
| How the score is computed | A small neural network | A dot product (or dot product with a weight matrix) |
| Which decoder state it uses | The previous step's state | The current step's state |
| Speed | Slower | Faster (just matrix multiplication) |

Transformers use a version of Luong's dot product, divided by $\sqrt{d_k}$ (the square root of the vector size), which stops the scores from getting too large.

---

## Q75. How should I understand Query, Key and Value?

**Simple answer**

Think of it as a **smart search**. The **Query** is what you're looking for. Each **Key** is a label describing an item. Each **Value** is the actual content of that item. Compare your query with every key to see how well they match, then return a **blend of the values**, weighted by how well each key matched.

**An analogy, a library search:**

- **Query:** "books about training neural networks"
- **Keys:** the titles on each book's spine
- **Values:** the actual contents of the books

You don't get just one book. You get a mix, mostly from the best-matching books.

```
Python dictionary:   d["cat"]                → exact match → exactly one value
Attention:           compare query with keys   → soft match  → weighted mix of all values
```

| Role | In Seq2Seq attention |
|---|---|
| Query | The decoder's current state ("what am I trying to write?") |
| Keys | The encoder outputs ("what does each input word contain?") |
| Values | The encoder outputs (often the same as the keys) |

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

In words: score every query against every key, turn the scores into weights, and use them to average the values.

**Looking ahead:** in *self-attention*, the queries, keys and values all come from **the same sentence**, so every word looks at every other word. Remove the RNN, keep only attention, and you have the core idea of the **Transformer** (next round).

---

### Q76. Explain why these architectures evolved: RNN → LSTM → Seq2Seq → Attention.

**Simple answer**

Each step fixed a specific weakness of the one before.

```
Normal network  ── no memory, fixed-size input
       ↓
RNN        → adds memory (hidden state), same weights every step
       ↓ problem: forgets things from long ago (vanishing gradients)
LSTM / GRU → gated memory that can keep information much longer
       ↓ problem: input and output can have different lengths
Seq2Seq    → encoder reads everything, decoder writes the answer
       ↓ problem: the whole input squeezed into one vector
Attention  → decoder looks back at every input word, at every step
       ↓ problem: RNNs still process one step at a time (slow)
[Transformer → drop the RNN, use only attention]   ← next round
```

---

## 23. Autoencoders and VAE

### Q77. What is an autoencoder, and why can't a plain autoencoder generate new data well?

**Simple answer**

An autoencoder **compresses** the input into a small code (the encoder), then tries to **rebuild** the original from that code (the decoder). It's trained only to make the rebuilt version match the original. It's useful for compression, removing noise and spotting anomalies. But the codes it learns are **scattered randomly**, with empty gaps between them. If you pick a random code and decode it, you usually get garbage.

```
image (784 numbers) → encoder → code (32 numbers) → decoder → rebuilt image (784 numbers)
training goal: rebuilt image ≈ original image
```

```
Plain autoencoder codes:          VAE codes:
  •3  •3        •7                  smooth, overlapping clouds,
         (empty gap → garbage)      packed around the centre, so
  •1   •1   •9      •0              any nearby point decodes into something sensible
```

**Anomaly detection use:** train an autoencoder only on **normal** data (say, readings from healthy machines). Later, if it rebuilds a new input badly, that input doesn't look like anything normal, so flag it as unusual.

---

## Q78. What problem does a VAE solve, and how does it differ from a plain autoencoder?

**Simple answer**

A **Variational Autoencoder (VAE)** organises the code space so we can **generate new data**. Instead of mapping each input to one exact point, the encoder outputs a **small cloud**: a centre ($\mu$) and a spread ($\sigma$). We pick a random point from that cloud and decode it. An extra penalty pulls all the clouds towards the centre of the space, so they overlap and leave no gaps. To create something new, pick a random point near the centre and decode it.

```
input ⟶ encoder ⟶ centre μ, spread σ ⟶ pick a random point z ⟶ decoder ⟶ output
```

| | Autoencoder | VAE |
|---|---|---|
| Encoder outputs | One point | A cloud (centre + spread) |
| Training goal | Rebuild the input | Rebuild the input + keep clouds near the centre |
| Code space | Gaps, scattered | Smooth and continuous |
| Can generate new samples? | Not reliably | Yes |
| Blending between two images | Often meaningless | Smooth, gradual change |

---

## Q79. Explain the VAE loss. Why do we need the KL-divergence term?

**Simple answer**

The VAE loss has two parts. The **reconstruction loss** asks: "did you rebuild the input well?" The **KL divergence** asks: "is your cloud close to a standard bell curve centred at zero?" Without the KL part, the encoder would make the clouds tiny and spread them far apart, which turns it back into a plain autoencoder with gaps. KL keeps the space tidy so random sampling works.

$$\text{VAE loss} = \text{Reconstruction loss} + \text{KL divergence}$$

**KL divergence** measures how different two distributions are. Here it compares each input's cloud with a standard normal distribution (centre 0, spread 1). It has a simple formula:

$$\text{KL} = -\frac{1}{2}\sum\left(1 + \log\sigma^2 - \mu^2 - \sigma^2\right)$$

You don't need to memorise it. Just know that it's **0 when $\mu = 0$ and $\sigma = 1$**, grows if the centre moves away from 0, and grows if the spread shrinks towards 0.

The tug-of-war:

```
Only reconstruction → tiny, far-apart clouds   (rebuilds well, but can't generate)
Only KL             → every input gives the same cloud   (tidy, but remembers nothing)
Both together       → clouds overlap enough to be smooth, but still carry information
```

**Interviewer may ask next**

- **"Why does the encoder output $\log \sigma^2$ instead of $\sigma$?"** The spread $\sigma$ must be positive, but a network's output can be any number. Predicting the log and then converting it back guarantees a positive value and is more stable.
- **"What is ß-VAE?"** Give the KL term extra weight (β > 1). This encourages each code number to control one clear property, such as rotation or thickness, at some cost to reconstruction quality.

---

## Q80. What is the reparameterization trick, and why is it needed?

**Simple answer**

Backprop can't pass through a **random sampling** step: you can't take the derivative of "pick a random number". The trick rewrites the sample as **centre + spread × random noise**: $z = \mu + \sigma \times \epsilon$, where $\epsilon$ is random noise drawn separately. Now the randomness is just an outside input, and $z$ is an ordinary formula of $\mu$ and $\sigma$, so gradients can flow through it.

$$z = \mu + \sigma \times \epsilon, \qquad \epsilon \sim \mathcal{N}(0, 1)$$

```
Before:   μ, σ ──► [ pick randomly ] ──► z        ✗ gradient can't pass through
After:    μ, σ ──► [ μ + σ × ε ] ──► z            ✓ gradient flows to μ and σ
                        ↑
                  ε = random noise (an input that doesn't need training)
```

**An analogy:** instead of "throw a dart somewhere near the target" (random and not differentiable), say "the dart lands at *target + spread × a dice roll*". The dice roll is random, but the formula around it is ordinary maths.

```
class Sampling(layers.Layer):
    def call(self, inputs):
        z_mean, z_log_var = inputs
        eps = keras.random.normal(shape=keras.ops.shape(z_mean))
        return z_mean + keras.ops.exp(0.5 * z_log_var) * eps      # σ = exp(½ · log σ²)
```

---

## Q81. Why are VAE samples often blurry? What is posterior collapse?

**Simple answer**

**Blurry:** the reconstruction loss (usually like MSE) is lowest when the model predicts the **average** of all the possible outputs. The average of several sharp images is a blurry image. **Posterior collapse:** if the decoder is very powerful, it can learn to **ignore the code** completely and generate on its own, so the code carries no information.

Fixes for collapse: start training with the KL weight at 0 and slowly increase it (**KL annealing**), or use a less powerful decoder.

Fixes for blur: add other losses (for example a GAN-style loss), or use improved VAE versions. Fun fact: Stable Diffusion still uses a VAE, but only to **compress** images. A different model (diffusion) does the actual generating.

---

# 24. Generative Adversarial Networks (GAN)

## Q82. What is the basic idea behind a GAN? Write the objective.

**Simple answer**

A GAN (2014) has **two networks competing**. The **generator** turns random noise into fake images. The **discriminator** looks at an image and guesses whether it's real or fake. The discriminator trains to catch fakes; the generator trains to fool it. As both get better, the fakes become very realistic. Interestingly, the generator **never sees real images**. It learns only from the discriminator's feedback.

**An analogy:** a counterfeiter (generator) and a police detective (discriminator). The detective gets better at spotting fake money, which forces the counterfeiter to make better fakes, and so on.

```
random noise ──► Generator ──► fake image ──┐
                                            ├──► Discriminator ──► "real or fake?"
                          real image ───────┘
          ▲                                              │
          └──── feedback: "make your fakes more convincing" ┘
```

The objective, written as a formula:

$$\min_G \max_D \; \mathbb{E}\big[\log D(\text{real})\big] + \mathbb{E}\big[\log(1 - D(G(\text{noise})))\big]$$

In plain words: the discriminator $D$ wants to output **high for real** and **low for fake**. The generator $G$ wants $D$ to output **high for its fakes**.

---

## Q83. How is a GAN actually trained? What is the non-saturating generator loss?

**Simple answer**

Training alternates two steps. **(1)** Train the discriminator on a batch of real images (label "real") and fake images (label "fake"). **(2)** Train the generator, keeping the discriminator fixed, to make the discriminator label its fakes as "real". In practice the generator's loss is slightly reworded (the "non-saturating" loss) so it still gets a strong learning signal early on, when its fakes are easy to spot.

```
repeat:
    # Step 1: train the discriminator
    real = some real images;  fake = G(random noise)
    loss_D = "real ones should score 1" + "fake ones should score 0"
    update D

    # Step 2: train the generator
    fake = G(random noise)
    loss_G = "my fakes should score 1"      ← the non-saturating loss
    update G only
```

**Why the rewording?** At the start, the discriminator easily rejects every fake (scores near 0). The original generator loss is almost flat in that region, so the generator gets almost no signal. Asking it to "make the discriminator say 1" instead gives a **steep, useful gradient** exactly when the generator needs it most.

---

## Q84. What does the discriminator learn, and what is the GAN really optimizing?

**Simple answer**

The discriminator learns the **probability that an image is real**. When the generator is perfect, its fakes match real data exactly and the best the discriminator can do is guess **0.5 for everything**. In effect, the discriminator works as a **learned loss function** that keeps improving its idea of what "real" looks like. That's why GAN images are sharp, unlike VAEs, which use a fixed MSE-like loss.

Mathematically, the GAN minimises a measure of how different the real and fake data distributions are (the Jensen–Shannon divergence). You only need to know the name.

**⚠ Common mistake**

If the discriminator becomes **too good too quickly**, it rejects every fake with total confidence, and the generator gets **almost no useful gradient** to learn from. This is a main cause of GAN training problems, and the motivation for **Wasserstein GAN (WGAN)**, which gives a useful signal even then.

---

## Q85. Why are GANs hard to train? What is mode collapse, and how do you fix these problems?

**Simple answer**

Training a GAN means balancing **two players**, not just making one loss go down, so it's unstable. Common problems: **mode collapse** (the generator only makes a few kinds of output), **the discriminator winning too easily** (the generator stops learning), **oscillation** (the two keep chasing each other without settling), and **losses that don't tell you if the images are good**.

**Mode collapse explained:** suppose you're generating handwritten digits. The generator discovers that its "1"s fool the discriminator, so it makes **only 1s**. It isn't directly rewarded for variety, only for fooling the discriminator.

| Problem | What you see |
|---|---|
| **Mode collapse** | All outputs look nearly the same |
| Discriminator too strong | Generator stops improving |
| Oscillation | Quality improves, then gets worse, over and over |
| Misleading losses | Loss numbers look fine, but the images are bad (or the reverse) |

| Fix | What it helps |
|---|---|
| Non-saturating generator loss | Generator gets a learning signal early on |
| **DCGAN design rules** (use strided convs, BatchNorm, LeakyReLU in the discriminator, tanh on the generator's output) | General stability for image GANs |
| **WGAN / WGAN-GP** (a different loss with a smoothness constraint) | Useful gradients even when the discriminator is winning; the loss actually tracks quality |
| **Spectral normalization** on the discriminator | Stops the discriminator from becoming too extreme |
| **Label smoothing** (use 0.9 instead of 1 for "real") | Stops the discriminator from becoming over-confident |
| Different learning rates for D and G | Balances the two players |

**Interviewer may ask next**

- **"How do you measure GAN quality?" FID (Fréchet Inception Distance)** compares statistics of real and generated images using a pretrained image network. **Lower is better.** It checks both quality and variety. And always look at the images yourself.
- **"What's a conditional GAN?"** You also give both networks a label, so you can control **what** is generated ("draw a 7"). Image-to-image versions turn sketches into photos (pix2pix) or horses into zebras (CycleGAN).

---

## 25. VAE vs GAN

### Q86. Compare VAE and GAN. When would you choose each?

**Simple answer**

**VAE:** stable and easy to train, has an encoder, and builds a smooth, well-organised code space, but its images are **blurrier**. **GAN:** produces **sharp, realistic** images, but training is unstable, it can lose variety (mode collapse), and it has no encoder by default. Choose a VAE for representations, smooth blending or anomaly detection. Choose a GAN when realistic images are what matters.

| | VAE | GAN |
|---|---|---|
| How it learns | Rebuild input + keep code space tidy | Generator vs discriminator game |
| Training | Stable: one loss that goes down | Unstable: needs careful tuning |
| Image quality | Smoother / blurrier | Sharp and realistic |
| Variety | Good: covers all kinds of data | Can collapse to a few kinds |
| Has an encoder (image → code)? | Yes | No (unless added) |
| Typical uses | Anomaly detection, compression, smooth blending | Realistic images, super-resolution, image-to-image translation |

```
VAE → organised code space   ("covers everything, a bit blurry")
GAN → realism through competition   ("sharp, but may miss some variety")
```

**Interviewer may ask next**

- **"What's used for image generation today?" Diffusion models.** They learn to remove noise from an image step by step. They give GAN-level quality with VAE-like stable training, but they're slower to generate. Stable Diffusion runs its diffusion model **inside a VAE's code space**, so the two ideas work together.

---

## 26. Debugging and Real-World Scenarios

> These questions show whether you've actually trained models. Answer with a **step-by-step checklist**, not a single guess.

## Q87. Your training loss isn't decreasing at all. How do you debug it?

**Simple answer**

First check whether the model **can learn at all**: try to make it **memorise one tiny batch** (10–50 examples) until the loss is near zero. If it can't even do that, there's a **bug** (in the data, labels, loss or code), not a model-size problem. Then check the learning rate.

### Checklist, in order:

1. **Look at your data.** Display a few images with their labels *after* preprocessing. Are they black? Wrongly scaled? Did the images and labels get shuffled separately?
2. **Check the starting loss.** With $K$ equally common classes, the first loss should be about $\ln K$ (2.3 for 10 classes). Something very different means a setup problem.
3. **Memorise one batch.** Turn off augmentation and dropout. If the loss won't go to near zero, there's a bug.
4. **Check that the loss matches the output.** Softmax with `from_logits=True`? One-hot labels with the `sparse_` loss? These mismatches are very common.
5. **Learning rate.** Try 10× bigger and 10× smaller.
6. **Gradients.** Are they all zero (dead neurons, frozen layers) or huge?
7. **Frozen by accident?** For example `trainable=False` on the whole model.
8. **Is the task possible?** Could a human do it from these inputs?

**⚠ Common mistake**

Don't jump straight to "use a bigger model" or "try another architecture". A correctly set-up small model should learn *something*. If nothing is learned, it's almost always a bug in the pipeline.

---

## Q88. The loss suddenly becomes NaN. What are the likely causes?

**Simple answer**

Usually either **exploding gradients** (often from a learning rate that's too high), or a **maths error** such as log(0), dividing by zero, or a number too large for the computer. Bad values (NaN or infinity) in the input data are another common cause.

| Cause | Fix |
|---|---|
| Learning rate too high | Lower it; add warmup |
| Exploding gradients (common in RNNs) | Gradient clipping ( `clipnorm=1.0` ) |
| log(0) in a hand-written loss | Use Keras's built-in losses; add a tiny $\epsilon$ |
| NaN or infinity in the data | Check with `np.isfinite(x).all()` |
| Pixels not scaled (0–255) | Scale to 0–1 |
| Dividing by zero in custom code | Add a tiny $\epsilon$ |

Useful tool: `keras.callbacks.TerminateOnNaN()` stops training immediately so you can investigate.

---

## Q89. You have only 1,500 labelled images for a defect-detection task. What's your plan?

**Simple answer**

Don't train from scratch. Use **transfer learning** from a pretrained model, use **sensible data augmentation**, split the data **carefully**, and start with a **simple baseline**. After that, improving the data usually helps more than a fancier model.

1. **Decide the task.** Just pass/fail → classification. Need the defect's location → detection. Need its size → segmentation (U-Net works well with small data).
2. **Split carefully.** Make sure photos of the **same part** don't end up in both train and validation, otherwise the scores will look falsely good.
3. **Baseline.** A pretrained ResNet-50 or EfficientNet with a new last layer. Then fine-tune the top layers with a small learning rate.
4. **Augmentation** that matches real-life variation: lighting, small rotations, blur. Avoid anything that could hide or create a defect.
5. **Imbalance:** defects are usually rare, so use class weights, and measure with **precision and recall**, not accuracy.
6. **Error analysis.** Look at the mistakes, then collect more of the kinds of examples the model gets wrong, and fix bad labels.
7. **If defects are very rare:** train only on good parts and flag anything that looks unusual (**anomaly detection**, for example with an autoencoder).
8. **More data ideas:** label the images the model is most unsure about first (**active learning**), or create synthetic defects.

---

## Q90. The model hits 95% on validation but performs badly in production. What could be wrong?

**Simple answer**

Suspect a **data difference** before blaming the model. Either the validation set didn't represent real-world data, or the real-world data has changed (**data drift**), or the images are **prepared differently** in production than in training.

| Category | Examples |
|---|---|
| **Different preprocessing** | RGB vs BGR colour order, different resizing, normalisation missing in production |
| **Real-world data is different** | New camera, lighting, angle or resolution; a different hospital's scanner |
| **Validation was too easy** | Near-duplicate images in both train and validation; the same patient in both |
| **Different class balance** | Defects were 10% of training data but are 0.1% in production |
| **Model learned a shortcut** | It learned something irrelevant that happened to correlate with the label, such as a hospital's watermark or a ruler placed next to cancer photos, and that clue isn't there in production |
| **Wrong threshold** | The decision cut-off was tuned on balanced data |

**What to do:** save real production inputs and predictions, compare them with the training data, use **Grad-CAM** (see Q95) to see what the model looks at, build a validation set **from real production data**, retrain with representative examples, and monitor for drift.

---

## Q91. Training accuracy is 99%, validation accuracy is 70%, and validation loss rises after epoch 5. What do you do?

**Simple answer**

That's classic **overfitting**: the model is memorising the training data. First make sure the data split is fine. Then use early stopping and the usual anti-overfitting tools, focusing first on **more or augmented data** and **transfer learning**.

1. Check the split (same kind of data in both, no duplicates).
2. **Early stopping** at the best epoch (`restore_best_weights=True`).
3. **Data augmentation**; collect more data if possible.
4. **Transfer learning** instead of training from scratch.
5. **Dropout**, **weight decay**, **label smoothing**.
6. **A smaller model**: fewer layers or filters, GAP instead of big dense layers.
7. Check the result: training accuracy may drop a little, and validation accuracy should rise. That's the trade you want.

**Interviewer may ask next**

- **"What if both training and validation accuracy are low?"** That's **underfitting**: the model is too simple or hasn't trained enough. Train longer, use a bigger model, reduce regularisation, check the learning rate, and check that the input actually contains the information needed.

---

## Q92. Your detector misses small objects. How would you improve it?

**Simple answer**

Small objects only cover a few pixels, and they **disappear** as the network keeps shrinking the image. Fix this by **keeping more detail**: use bigger input images, and use a detector that predicts from the high-resolution early layers too. Also tune the anchors, augmentation and NMS for small boxes.

- Use a **larger input size** (e.g. 640 → 1280), or **cut big images into overlapping tiles** (common for drone and satellite images).
- Use a detector that predicts at **several scales**, and make sure the finest scale is used.
- **Re-fit the anchor sizes** to your dataset's box sizes.
- **Augmentation:** copy-paste extra small objects, zoom in.
- **Check the labels:** small objects are the ones annotators miss most.
- Measure progress with the **small-object score** specifically (COCO reports AP for small, medium and large objects).

---

## Q93. You need to run an image model on a phone at 30 FPS. What would you change?

**Simple answer**

Use an **efficient architecture** (MobileNet, EfficientNet-Lite or a small YOLO), use **smaller input images**, and **compress** the trained model, mainly with **quantization** (storing numbers with fewer bits). Always measure speed **on the actual phone**: parameter counts are only a rough guide.

| Change | Typical effect |
|---|---|
| **Depthwise-separable convs** (MobileNet) | About 8–9× less computation than normal convs |
| Smaller input (224 → 160 pixels) | About 2× less computation |
| Fewer channels per layer | Computation drops quickly |
| **8-bit quantization** (TFLite) | About 4× smaller model, often 2–4× faster, usually under 1% accuracy loss |
| **Pruning** | Removes unimportant weights |
| **Knowledge distillation** | Train a small "student" model to copy a big "teacher" model |
| Phone hardware acceleration | Run on the phone's GPU or AI chip |

---

## Q94. Your image classes are heavily imbalanced (95% / 5%). How do you handle it?

**Simple answer**

First, **use the right metric**: precision, recall, F1 and the confusion matrix, **not accuracy**. A model that always predicts the big class already gets 95% accuracy while being useless. Then help the model pay attention to the small class: **class weights** or **focal loss**, **oversampling** the small class (with augmentation), and **adjusting the decision threshold** based on which mistakes cost more.

```
model.fit(train_ds, class_weight={0: 1.0, 1: 19.0}, ...)   # rare class counts ~19× more
```

Also make sure the validation set contains enough rare examples to measure anything (use a **stratified** split).

---

## Q95. How would you explain *why* a CNN made a particular prediction?

**Simple answer**

Use a method that highlights **which parts of the image led to the prediction**. The most common is **Grad-CAM**. It produces a heatmap over the image showing where the model was "looking" when it decided on a class.

**How Grad-CAM works, in plain steps:**

1. Take the last conv layer's feature maps.
2. Calculate how much each feature map affects the score for the chosen class (using gradients).
3. Combine the feature maps, weighting each by its importance.
4. Keep only the positive parts and resize the result to the image size → a heatmap.

**Why it's valuable:** it catches models that learned the **wrong thing**. For example, a "husky vs wolf" classifier that is actually just detecting **snow** in the background (wolves were usually photographed in snow), or a medical model looking at the hospital's text label instead of the lungs.

Other methods: cover parts of the image and see how the prediction changes (occlusion), Integrated Gradients, and SHAP.

---

# 27. Rapid-Fire Questions

> Short answers in one or two lines. Interviewers use these to quickly test how broad your knowledge is.

**Q96. Why can't you use a step function as the activation when training?**
Its slope is zero almost everywhere, so backprop gets no gradient to learn from.

**Q97. What's the slope of ReLU at exactly 0?**
Technically undefined. Libraries just use 0, and in practice it doesn't matter.

**Q98. Do pooling layers have parameters?**
No. They are fixed operations (take the max or the average).

**Q99. Does a BatchNorm layer have parameters?**
Yes: a learned scale $\gamma$ and shift $\beta$ per channel, plus saved running averages (not trained).

**Q100. Why is softmax paired with categorical cross-entropy?**
Together they're the mathematically natural choice for classification, and the gradient becomes simply "prediction – truth".

**Q101. What happens if the learning rate is too high? Too low?**
Too high: the loss jumps around or becomes NaN. Too low: training is very slow or gets stuck.

**Q102. Epoch vs batch vs iteration?**
Epoch = one full pass over the data. Batch = the examples used in one update. Iteration = one update. Iterations per epoch = number of examples ÷ batch size.

**Q103. Why shuffle the training data every epoch?**
So every batch is a fair mix, and the model doesn't learn anything from the order of the data.

**Q104. Is dropout used when making predictions?**
No. It's only active during training.

**Q105. Why can training loss be higher than validation loss?**
Dropout and augmentation make training deliberately harder, and they're switched off for validation.

**Q106. Can a CNN take images of different sizes?**
The conv layers can. A Flatten + Dense ending can't. Use Global Average Pooling to make the whole network accept any size.

**Q107. Why do CNNs usually double the channels when they halve the image size?**
To keep the amount of computation per layer roughly balanced, swapping position detail for richer features.

**Q108. Pooling vs stride-2 convolution?**
Both shrink the image. Stride-2 conv is learned (more flexible); pooling is fixed and free.

**Q109. Are CNNs rotation-invariant?**
No. Robustness to rotation has to come from data augmentation.

**Q110. What area do three stacked 3×3 convs (stride 1) see?**
7×7.

**Q111. Why are 3×3 filters so popular?**
It's the smallest size that captures left/right, up/down and centre. Stacking them is cheaper and more powerful than one large filter.

**Q112. ResNet-34 blocks vs ResNet-50 blocks?**
ResNet-34 uses basic blocks (two 3×3 convs). ResNet-50 uses bottleneck blocks (1×1 → 3×3 → 1×1).

**Q113. Why are GoogLeNet's helper classifiers removed at prediction time?**
They only gave extra learning signal during training.

**Q114. Can you use an ImageNet-pretrained model on grayscale X-rays?**
Yes. Copy the single channel three times to make it look like RGB. The pretrained features still help.

**Q115. In detection, what counts as a correct box?**
A box with IoU ≥ 0.5 (usually) with a real box of the same class that hasn't already been matched.

### Q116. Why does YOLO predict the square root of width and height?
So the same pixel error counts more on small boxes than on big ones.

### Q117. What does IoU = 0.5 look like?
Less overlap than you'd guess: two same-size boxes must share two-thirds of their area to reach 0.5.

### Q118. Does NMS learn anything?
No. It's a fixed clean-up step with a threshold you choose.

### Q119. Why do RNNs use tanh instead of ReLU?
tanh keeps values between –1 and 1, so the memory doesn't blow up when the same weights are applied over and over.

### Q120. What does `return_state=True` give you in a Keras LSTM?
The final output state and cell state. You pass these to a Seq2Seq decoder to start it.

### Q121. Why is the LSTM forget-gate bias set to 1 at the start?
So the gate starts out "remembering", which helps gradients flow from the beginning.

### Q122. LSTM or GRU for a small dataset?
Often GRU (fewer parameters, less overfitting), but try both.

### Q123. What's the `<END>` token for?
It lets the decoder say "I'm done", so outputs can be any length.

### Q124. What does an attention weight of 0.9 on one input word mean?
At that step, the decoder is mostly "looking at" that word, usually the word it's translating.

### Q125. Why do we sample from a standard normal distribution to generate with a VAE?
Because the KL term trained the code space to look like that distribution.

### Q126. What should a perfect GAN's discriminator output?
0.5 for every image, because it can't tell real from fake.

### Q127. Is a falling generator loss a sign of a good GAN?
Not necessarily. GAN losses depend on the opponent. Look at the images and track FID.

### Q128. What is mode collapse, in one sentence?
The generator keeps producing only a few kinds of output instead of the full variety of real data.

### Q129. GAN or VAE for anomaly detection?
Usually a VAE or autoencoder. They have an encoder and a reconstruction error you can set a threshold on.

---

## Formula Cheat Sheet

| What | Formula | Example |
|---|---|---|
| Dense layer params | $(n_{in} + 1) \times n_{out}$ | 784 → 128: 100,480 |
| Conv output size | $\frac{W-K+2P}{S} + 1$ (round down) | 224, K=7, S=2, P=3 → 112 |
| Conv params | $(K \times K \times C_{in} + 1) \times C_{out}$ | 3×3, 64 → 128: 73,856 |
| Conv multiply-adds | $H_{out} \times W_{out} \times C_{out} \times K^2 \times C_{in}$ | Inception 5×5 example: 120M |
| Depthwise-separable params | $K^2 C_{in} + C_{in} C_{out}$ | ~8–9× fewer for 3×3 |
| Transposed conv output | $(W - 1) \times S + K$ | 4, S=2, K=2 → 8 |
| Receptive field | new RF = old RF + $(K - 1) \times j$ | three 3×3 → 7 |
| Simple RNN params | $h(h + d) + h$ | $d$=100, $h$=64: 10,560 |
| LSTM params | $4 \times [h(h + d) + h]$ | $d$=100, $h$=128: 117,248 |
| GRU params | about $3 \times [h(h + d) + h]$ | ~88K |
| Softmax | $\frac{e^{z_k}}{\sum_j e^{z_j}}$ | [2, 1, 0.1] → [0.66, 0.24, 0.10] |

| What | Formula | Example |
|---|---|---|
| Cross-entropy gradient | prediction – truth | |
| He / Xavier variance | $\frac{2}{n_{in}}$ / $\frac{2}{n_{in}+n_{out}}$ | |
| IoU | overlap ÷ (area A + area B – overlap) | 25 / 175 = 0.14 |
| YOLOv1 output | $S \times S \times (5B + C)$ | 7 × 7 × 30 |
| Attention | $\text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$ | |
| VAE KL | $-\frac{1}{2}\sum(1 + \log\sigma^2 - \mu^2 - \sigma^2)$ | 0 when $\mu$=0, $\sigma$=1 |
| Dice | $\frac{2\times\text{overlap}}{\text{area A}+\text{area B}}$ | |

## Rapid Revision Table

| Topic | Key idea in one line | Number to remember |
|---|---|---|
| Perceptron | Draws one straight line | Can't solve XOR |
| MLP | Layers + nonlinearity = complex functions | 1 hidden layer is enough *in theory* |
| Sigmoid | Squashes to 0–1, flattens out | Max slope 0.25 |
| ReLU | $\max(0, x)$; slope 1 when active | Can "die" |
| Cross-entropy | Loss for classification | Gradient = prediction – truth |
| Backprop | Chain rule, going backwards | |
| He init | Weight scale for ReLU | Keras default is Glorot |
| Adam | Momentum + per-weight step sizes | LR 0.001 |
| Dropout | Randomly switch off neurons in training | Off at prediction time |
| BatchNorm | Re-scale values per channel | Uses saved averages at prediction time |
| CNN | Small shared filters | Params don't depend on image size |
| 1×1 conv | Mixes channels at each pixel | ~10× cheaper Inception |
| LeNet-5 | First practical CNN recipe | ~60K params, 1998 |
| AlexNet | CNN at scale with ReLU, dropout, GPUs | ~60M params, 15.3% top-5 (2012) |
| VGG-16 | Only 3×3 filters, deep | ~138M params, 89% in dense layers |
| GoogLeNet | Several filter sizes in parallel | ~6–7M params, 6.67% top-5 |
| ResNet | output = F(x) + x | 152 layers, 3.57% top-5 (2015) |
| ResNet-50 | Bottleneck blocks, stages 3-4-6-3 | ~25.6M params |
| Transfer learning | Reuse a pretrained model | Fine-tune with a 10–100× smaller LR |
| Detection | Label + box for each object | Scored with mAP |
| YOLOv1 | Grid, one pass | 7×7×30 output, 45 FPS |
| IoU | Overlap ÷ union | 0.5 = usual "correct" threshold |
| NMS | Keep the best box, delete overlapping ones | Threshold ~0.5 |
| U-Net | Shrink, then grow back + skip connections | Works with small datasets |
| RNN | Memory + same weights each step | Forgets long-range info |
| LSTM | Conveyor-belt memory + 3 gates | 4× RNN params |
| GRU | Simpler: 2 gates | 3× RNN params |
| Seq2Seq | Encoder → summary → decoder | One-vector bottleneck |
| Attention | Look back at all inputs at every step | Weights from softmax |
| VAE | Clouds instead of points + KL | $z = \mu + \sigma\epsilon$ |
| GAN | Generator vs discriminator | Perfect D outputs 0.5 |

## Architecture Map

```
ANN (basic neural network)
├── Neurons, hidden layers, width vs depth
├── Activations (sigmoid → ReLU)
├── Losses (MSE, cross-entropy)
└── Training: backprop, initialisation, optimizers, dropout, BatchNorm
        │
        ▼
CNN (small shared filters that build up from edges to objects)
├── LeNet-5 → AlexNet → VGG → GoogLeNet/Inception → ResNet / ResNet-50
│                                                          │
│                                    used as the base for ─┤
│                                                          ▼
├── Vision tasks
│   ├── Classification
│   ├── Detection ── two-stage (Faster R-CNN) / one-stage (YOLO)
│   │                IoU · NMS · anchors · mAP
│   └── Segmentation ── U-Net · semantic / instance / panoptic
│
Sequences
├── RNN → LSTM / GRU → Seq2Seq → Attention → [Transformers: next round]
│
Generative models
├── Autoencoder → VAE
└── GAN → DCGAN, WGAN, conditional GANs
        └── (both lead to) → Diffusion models
```

## What a Strong Candidate Shows

- **Explains *why*, not just *what*.** Every architecture fixed a specific weakness of the one before.
- **Can calculate on the spot.** Output sizes, parameter counts, receptive fields, IoU.
- **Knows what changes between training and prediction.** Dropout, BatchNorm, teacher forcing, augmentation.
- **Debugs step by step.** Memorise one batch; check the data, the starting loss, the learning rate and the gradients before changing the model.
- **Connects choices to the real problem.** Pick the task (classification, detection or segmentation) based on the business decision; pick augmentations that don't change labels; pick metrics based on class balance.
- **Talks about trade-offs honestly.** Depth vs ease of training, accuracy vs speed, GAN sharpness vs stability.
- **Avoids absolute statements.** "Deeper is always better" and "deep learning always beats trees" are both wrong.

## Core Mental Model

```
Don't memorise:
"LeNet has these layers, AlexNet has these layers, VGG has these layers…"

Instead, for any architecture, be able to answer:

  What problem existed?
        ↓
```

```
What was wrong with the previous model?
      ↓
What new idea fixed it?
      ↓
What did that new idea cost?
      ↓
What problem was left, and what came next?
```

If you can follow this chain of reasoning, you can answer questions you've never seen before, not just the ones you practised.